

dapper: Data Augmentation for Private Posterior Estimation in R

by Kevin Eng, Jordan A. Awan, Nianqiao Phyllis Ju, Vinayak A. Rao, and Ruobin Gong

Abstract This paper serves as a reference and introduction to using the R package `dapper`. `dapper` encodes a sampling framework which allows exact Markov chain Monte Carlo simulation of parameters and latent variables in a statistical model given privatized data. The goal of this package is to fill an urgent need by providing applied researchers with a flexible tool to perform valid Bayesian inference on data protected by differential privacy, allowing them to properly account for the noise introduced for privacy protection in their statistical analysis. `dapper` offers a significant step forward in providing general-purpose statistical inference tools for privatized data.

1 Introduction

Differential privacy (DP) provides a rigorous framework for protecting confidential information from re-identification attacks by using random noise to obscure the connection between the individual and their data (Dwork et al., 2006b). Its development was spurred on by successful attacks on anonymized data sets containing sensitive personal information. Prior to differential privacy, anonymization schemes did not always have sound theoretical guarantees despite appearing adequate. Differential privacy marks a leap forward in the science of privacy by placing privacy guarantees on a rigorous theoretical footing and moving beyond earlier ad hoc approaches. Several recent high-profile implementations of differential privacy include those by Apple (Tang et al., 2017), Google (Erlingsson et al., 2014), Microsoft (Ding et al., 2017), and the U.S. Census Bureau (Abowd, 2018).

Many data sets amenable to differential privacy contain valuable information that stakeholders are still interested in learning about. However, the noise introduced by differential privacy changes the calculus of inference. As an example, we can implement differential privacy for tabular data by directly adding independent, random error to each cell, the amount and type of which are determined by the DP mechanism design. When we fit a regression model to the noise-infused data, this will correspond to having measurement errors in the covariates. This, unfortunately, violates the assumptions of most statistical models. In the presence of such errors, standard estimators can exhibit significant bias and incorrect uncertainty quantification (Gong, 2022; Karwa et al., 2015; Wang et al., 2018). These issues are a serious concern for researchers (Santos-Lozada et al., 2020; Kenny et al., 2021; Winkler et al., 2022). Given the importance of statistical methods in scientific research, the increased adoption of privacy tools makes developing privacy-aware statistical workflows crucial.

Unfortunately, making the necessary adjustments poses formidable mathematical challenges (Williams and McSherry, 2010), even for seemingly simple models like linear regression. The difficulty lies in the marginal likelihood for the privacy-protected data. This function is often analytically intractable, and, as a result, it is difficult or impossible to apply traditional statistical methods to derive estimators. In particular, the marginal likelihood can involve a complex integral where it is not even possible to evaluate the likelihood at a point. Tackling the problem by approximating the likelihood can be computationally infeasible since the integral is usually high dimensional. Few tools are available to researchers to address these issues, and their absence is a serious barrier to the wider adoption of differential privacy.

The `dapper` package, short for Data Augmentation for Private Posterior Estimation in R, provides a set of tools for conducting privacy-aware Bayesian inference. It serves as an R interface for the data augmentation MCMC framework proposed by Ju et al. (2022), allowing existing Bayesian models to be extended to handle noise-infused data. The package is designed to integrate well with existing Bayesian workflows; results can be analyzed using tools from the `rstan` (Stan Development Team, 2025) ecosystem in a drop-in fashion. Additionally, construction of a privacy-aware sampler is simplified through the specification of four independent modules: `latent_f()`, `posterior_f()`, `statistic_f()`, and `mechanism_f()`, which are described in detail later in the paper. The benefits are twofold: several privacy mechanisms — these can even be from different formal privacy frameworks — can be compared easily by only swapping out relevant modules. Furthermore, privacy mechanisms that have non-smooth transformations resulting in the aforementioned intractable likelihoods (see Example 3, which involves clamping) can be incorporated with little work. As a result, `dapper` may prove particularly useful to those engaged in studying the privacy-utility trade-off or dealing with a privacy mechanism that involves multiple transformations.

1.1 Related software and paper outline

Software for differential privacy spans two distinct tasks: constructing privacy-preserving releases and performing valid inference after a private release has been observed. Table 1 compares **dapper** with the R packages **tidydp** (Tarler, 2025), **DPpack** (Giddens, 2024), **diffpriv** (Rubinstein and Aldà, 2017), and **SimBaRepro** (Du et al., 2025); the multilingual OpenDP library (The OpenDP Project, 2026); and the Python packages **d3p** (Prediger et al., 2022) and **diffprivlib** (Holohan et al., 2019).

Table 1: Comparison of representative differential-privacy software. Rows are ordered by decreasing similarity to `\CRANpkg{dapper}`, prioritizing support for inference from an existing private release, followed by inferential paradigm and scope. The "Existing private release?" column indicates whether a tool performs inference conditional on an already released privatized statistic or data set; "Limited" denotes support restricted to a specialized setting. VI denotes variational inference. Among the tools compared, only **dapper** performs Bayesian inference from existing private releases with user-defined models and privacy mechanisms.

Software	Language	Role and paradigm	"Existing private release?"	Scope and computation
dapper	R	Inference; Bayesian MCMC	Yes	User-defined models and mechanisms; latent-data MCMC
SimBaRepro	R	Inference; frequentist	Yes	Simulation-based tests and confidence sets; no point estimate
OpenDP	R, Python, Rust	Private release framework; specialized inference	Limited	Composable mechanisms; specialized binomial inference
d3p	Python	Private fitting; Bayesian VI	No	NumPyro models; minibatched JAX on CPU/GPU
DPpack	R	Private release and fitting; frequentist	No	Statistics, regression, and classification; model-specific
diffprivlib	Python	Private release and fitting; frequentist	No	Mechanisms, accounting, and scikit-learn-style estimators
diffpriv	R	Private release; not inferential	No	Generic mechanisms and sensitivity sampling; target-dependent
tidydp	R	Private release; not inferential	No	Tidy counts, sums, and means; in-memory data frames

The closest inferential comparison is **SimBaRepro** (Du et al., 2025), an R implementation of the simulation-based, finite-sample inference framework developed by Awan and Wang (2025). It also operates on an existing private release when the privacy mechanism is known, but it provides finite-sample frequentist tests, confidence intervals, and confidence sets rather than a point estimate or posterior draws. The **d3p** package also provides Bayesian methodology, but makes model fitting differentially private rather than conditioning on a previously released private statistic. The remaining tools primarily construct private releases or fit models directly to confidential data under a privacy guarantee. OpenDP also provides specialized inference utilities, such as confidence intervals and hypothesis tests for binomial statistics released with its canonical noise mechanism, but it does not provide general-purpose inference from arbitrary privatized statistics.

The rest of this article is organized as follows: Section 2 covers the necessary background to understand the mathematical notation and ideas used throughout the paper. Section 3 goes over the main algorithm without going into mathematical detail; for specifics, see Ju et al. (2022). Section 4 provides an overview of the **dapper** package and discusses important implementation details. Section 5 contains four examples of how one might use the package to analyze the impact of adding noise for privacy. The first example goes over a typical odds ratio analysis for a 2×2 table, the second example highlights the modular nature of **dapper** and reanalyzes the first example under a different privacy scheme, the third example covers a linear regression model, and the fourth example shows how **dapper** can be used as a wrapper around an existing sampler. Finally, Section 6 reports empirical performance for the examples and discusses an important practical implication of using a small privacy loss budget with **dapper**.

2 Background

Let $x = (x_1, \dots, x_n) \in \mathcal{X}^n$ represent a confidential database containing n records. We assume that the records are independent and identically distributed (i.i.d.) draws from a statistical model $f(\cdot \mid \theta)$, though this assumption can be relaxed. In many studies, scientists are interested in learning about θ because it provides important information about the scientific question of interest.

In the Bayesian statistical framework, learning about θ is accomplished by using the data x to update the posterior $p(\theta \mid x) \propto f(x \mid \theta)p(\theta)$. Here, $p(\theta)$ is called the prior distribution, and represents the researcher’s belief about θ before seeing any data. The posterior represents uncertainty around θ and is formed by using Bayes’ rule to fuse together the observed data and the researcher’s prior belief. One major advantage of the Bayesian method is that, through the prior, it provides a mechanism for

incorporating information not explicitly contained in the data at hand. This is especially useful in settings where there is considerable domain knowledge about the value of θ .

When they are available, it is desirable to work with a summary statistic $s = s(x)$ that has much smaller dimension than the original data because doing so can greatly simplify calculations. Since summary statistics are easier to work with, database curators often publish them to efficiently communicate information contained in large data sets. This makes them a natural target for dissemination-based privacy approaches (Karr, 2016).

2.1 Differential privacy

While a summary statistic can already partially anonymize data, it is still possible to deduce information about an individual depending on how x is distributed. Differential privacy offers a more principled approach by introducing randomness such that the output distribution does not change much when one individual's data is changed. A common approach — and the one **dapper** is primarily designed to address — is to take a summary statistic s , and add noise to it to produce a noisy summary statistic s_{dp} .

While adding noise to confidential data is already a well-established practice in statistical disclosure control (Dalenius and Reiss, 1982), differential privacy provides a rigorous framework to specify where and how much noise to add. Most importantly, for the analyst, the specification of the differentially private noise mechanism can be made available without compromising privacy and thus incorporated into subsequent analyses.

The **dapper** package provides a flexible framework that can accommodate the many different flavors of differential privacy. The main requirement is that the privacy mechanism's log density or log probability mass can be evaluated. However, for presentation, in this section we focus on the earliest and most common formulation of differential privacy, ϵ -differential privacy (ϵ -DP). The ϵ parameter is called the privacy loss budget. This parameter controls how strong the privacy guarantee is. Larger values of ϵ correspond to weaker privacy guarantees which in turn means less noise being added.

We now describe the ϵ -DP framework in more detail. For the noisy summary statistic, we write $s_{dp} \sim \eta(\cdot | x)$. Here, η denotes the density of the privacy mechanism. The mechanism is said to be ϵ -differentially private (Dwork et al., 2006b) if for all values of s_{dp} and all “neighboring” databases $(x, x') \in \mathcal{X}^n \times \mathcal{X}^n$ differing by one record (specifically, we consider $d(x, x') \leq 1$ where d is the Hamming distance), the probability ratio is bounded:

$$\frac{\eta(s_{dp} | x)}{\eta(s_{dp} | x')} \leq \exp(\epsilon), \quad \epsilon > 0.$$

The differential privacy framework is used to create and verify privacy mechanisms. One such mechanism is the *Laplace mechanism*. Given a statistic $s : \mathcal{X}^n \rightarrow \mathbb{R}^m$, the Laplace mechanism constructs the privatized statistic

$$s_{dp} := s(x) + u,$$

where the coordinates of u are independent Laplace random variables. Define the ℓ_1 -global sensitivity of s as

$$\Delta_1(s) := \max_{\substack{(x, x') \in \mathcal{X}^n \times \mathcal{X}^n \\ d(x, x') \leq 1}} \|s(x) - s(x')\|_1.$$

If each $u_j \sim \text{Lap}(0, \Delta_1(s)/\epsilon)$, then s_{dp} is ϵ -differentially private (Dwork et al., 2006b). Example 3 applies the Laplace mechanism to linear regression. Other common noise-adding mechanisms include the Gaussian and the discrete Gaussian mechanisms, which also add noise scaled to the sensitivity.¹

¹The Gaussian mechanisms require different and more general notions of DP than ϵ -DP, in particular, zCDP (Bun and Steinke, 2016) and (ϵ, δ) -DP (Dwork et al., 2006a). Example 2 will consider (ϵ, δ) -differential privacy, which extends ϵ -differential privacy to the case where the ratio bound can fail with probability governed by δ . More specifically, we say a privacy mechanism, $\mathcal{M}$, satisfies (ϵ, δ) -differential privacy if for all neighboring databases where $d(x, x') \leq 1$, we have

$$P(\mathcal{M}(x) \in S) \leq \exp(\epsilon)P(\mathcal{M}(x') \in S) + \delta$$

for any $S \subseteq \text{Range}(\mathcal{M})$ and $\delta \in [0, 1]$. Note that setting $\delta = 0$ gives us back the pure ϵ -differential privacy condition.

3 Methodology

Given privatized data, s_{dp} , the goal of Bayesian inference is to learn the private posterior distribution $p(\theta \mid s_{dp})$. Since the observed likelihood, $p(s_{dp} \mid \theta)$, often has no simple closed-form expression (Williams and McSherry, 2010), most standard approaches do not apply. To conduct privacy-aware Bayesian inference, the **dapper** package implements the data augmentation algorithm introduced in Ju et al. (2022), which allows us to sample from $p(\theta \mid s_{dp})$ without knowing a closed-form expression proportional to $p(s_{dp} \mid \theta)$. The algorithm considers the joint distribution $p(\theta, x \mid s_{dp})$ and alternates between sampling from the two conditional distributions $p(\theta \mid x, s_{dp})$ and $p(x \mid \theta, s_{dp})$.

Since s_{dp} is derived from x , we have $p(\theta \mid x, s_{dp}) = p(\theta \mid x)$, which is the usual posterior distribution given the confidential data x . The **dapper** package assumes that the user has access to a sampler for $p(\theta \mid x)$. This sampler may either draw directly from $p(\theta \mid x)$, such as under posterior conjugacy, or apply an MCMC transition kernel that preserves $p(\theta \mid x)$. For the second conditional distribution, $p(x \mid \theta, s_{dp})$ may only be known up to a constant. The **dapper** package samples from this distribution by running a Gibbs-like sampler: similar to a Gibbs sampler, each of the n components of x is individually updated. However, unlike a standard Gibbs sampler, each component is updated using a Metropolis–Hastings algorithm. This method is sometimes called a Metropolis-within-Gibbs sampler (Robert and Casella, 2004).

In some cases, sampling from $p(x \mid \theta, s_{dp})$ can be made more efficient when the privacy mechanism can be written as a function of s_{dp} and a sum consisting of contributions from each individual record. More precisely, we say the privacy mechanism satisfies the *record-additivity* property if

$$\eta(s_{dp} \mid x) = g\left(s_{dp}, t(x, s_{dp})\right) = g\left(s_{dp}, \sum_{i=1}^n t_i(x_i, s_{dp})\right),$$

for some known and tractable functions $g, t_1, \dots, t_n$ where $t(x, s_{dp}) := \sum_{i=1}^n t_i(x_i, s_{dp})$. The sample mean is an example of a summary statistic satisfying record-additivity where $t_i(x_i, s_{dp}) = x_i$. Record-additivity is not required for correctness. However, it allows the statistic to be updated in $O(1)$ time after replacing a single record, rather than recomputed over all n records.

The following pseudocode shows how to generate iteration $t + 1$ from iteration t in the data augmentation algorithm:

1. Sample θ^{t+1} from $p(\cdot \mid x^{(t)})$.
2. Sample from $p(x \mid \theta^{t+1}, s_{dp})$ using a three-step process:
 - Propose $x_i^* \sim f(\cdot \mid \theta^{t+1})$.
 - Assuming s satisfies the record-additive property, perform the update $t(x^*, s_{dp}) = t(x, s_{dp}) - t_i(x_i, s_{dp}) + t_i(x_i^*, s_{dp})$.
 - Accept the proposed state with probability $\alpha(x_i^* \mid x_i, x_{-i}, \theta^{t+1})$ given by:

$$\begin{aligned} \alpha(x_i^* \mid x_i, x_{-i}, \theta^{t+1}) &= \min \left\{ 1, \frac{\eta(s_{dp} \mid s(x_i^*, x_{-i}))}{\eta(s_{dp} \mid s(x_i, x_{-i}))} \right\} \\ &= \min \left\{ 1, \frac{g(s_{dp}, t(x^*, s_{dp}))}{g(s_{dp}, t(x, s_{dp}))} \right\}. \end{aligned}$$

Theoretical results such as bounds on the acceptance probability as well as results on ergodicity can be found in Ju et al. (2022).

4 The structure of dapper

The **dapper** package is structured around the two functions `dapper_sample()` and `new_privacy()`. The function `dapper_sample()` is used to generate MCMC draws from the private posterior. Since constructing the private posterior requires a large set of inputs, the process of setting up the sampler uses a privacy S3 object to encapsulate all information about the data-generating process. The role of `new_privacy()` is to construct privacy objects. This separates inputs describing the data-generating process from inputs describing simulation parameters, which decreases the chance for input-related bugs.

Utility functions facilitating work with count data are also included. These center around the probability mass function and random number generators of the discrete Gaussian and discrete Laplace distributions and are described in more detail in Section 4.4.

4.1 Software architecture

Figure 1 summarizes the typical **dapper** workflow. The four user-defined components describe the statistical model and privacy mechanism and are validated and collected by `new_privacy()` in a privacy S3 object. This separation keeps the model specification independent of the observed private release and the MCMC controls, allowing a component such as `mechanism_f()` to be replaced without rewriting the remainder of the analysis.

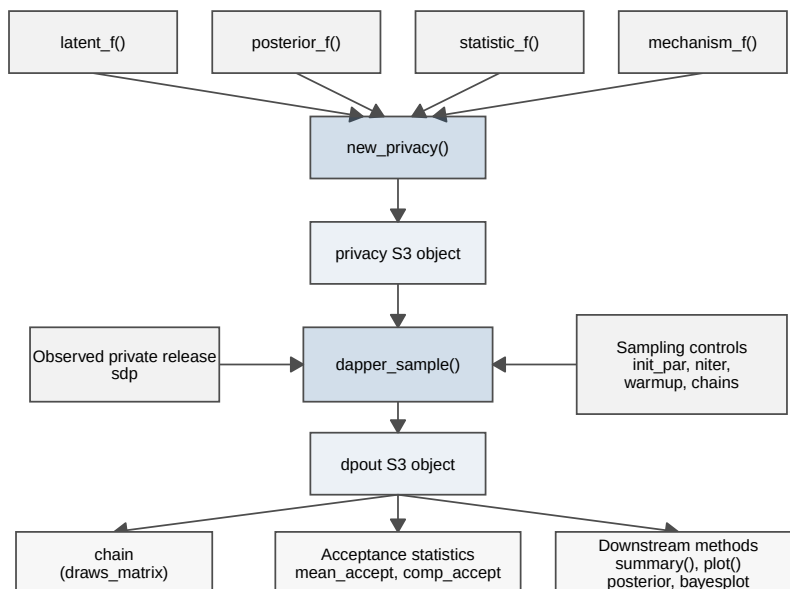


Figure 1: Software architecture and typical workflow of **dapper**. User-defined model components are assembled into a privacy object, combined with the observed private release and sampling controls by the main sampler, and returned as a dpout object for posterior analysis and diagnostics.

The `dapper_sample()` function combines the privacy object with the observed release `sdp`, an initial parameter value, and the requested simulation settings. It returns a dpout S3 object containing posterior draws and acceptance-rate diagnostics. Its chain component is a `draws_matrix`, so the result can be summarized and visualized using the package’s S3 methods or passed directly to **posterior** and **bayesplot**.

4.2 Privacy model

Creating a privacy model is done using the `new_privacy()` constructor. The main arguments consist of the four components as outlined in the methodology section.

```
new_privacy(posterior_f = NULL, latent_f = NULL, mechanism_f = NULL,
            statistic_f = NULL, npar = NULL, varnames = NULL)
```

To minimize the potential for bugs, the four main components must meet the requirements described below:

- `posterior_f()` is a function which makes a one-step draw from the posterior given the imputed confidential data. It has the syntax `posterior_f(dmat, theta)`. Here `dmat` is a numeric matrix representing the confidential database and `theta` is a numeric vector which serves as the initialization point for a one-sample draw. The simplest case is when a conjugate prior yields a posterior from which `posterior_f()` can draw independent parameter samples. If such a prior does not exist or is not appropriate, then `posterior_f()` can be one or more steps of any Markov chain that starts with the current parameter value, and has this posterior as its stationary distribution. It is tempting to construct this by wrapping an MCMC sampler generated from other R packages (e.g., **rstan**, **fmcmm** (Vega Yon and Marjoram, 2019), **adaptMCMC** (Scheidegger, 2024)); however, this requires caution. Such package samplers often use the initial burn-in samples to tune MCMC parameters, so that the resulting map from the input of `posterior_f()` to its output no longer satisfies detailed balance (concretely, we require that if the current parameter sample comes from the posterior, then so does the output of `posterior_f()`). Additionally,

these samplers can have a large initialization overhead and implicit logic that makes reasoning about correctness difficult under little or no burn-in. For example, it is not immediately clear from the [rstan](#) user manual how a sampler behaves when using no burn-in. For these reasons, we recommend sticking with conjugate priors when possible, as they will be quick and avoid serious undetected semantic errors that arise from specific implementation details of other R packages. If a non-conjugate prior is required, we recommend constructing `posterior_f()` using a lightweight, well-documented package such as [fmcmm](#). When `posterior_f()` uses an MCMC sampler rather than a direct conjugate draw, each call may perform one or more transitions targeting the conditional posterior. This may include an adaptive sampler, provided that its adaptation scheme is valid for the overall MCMC algorithm. The computational implications of choosing the number of inner transitions are discussed in Section 6.1.

- `latent_f()` is a function that generates data from the latent model. Its syntax must be `latent_f(theta)`, where `theta` is a numeric vector representing the model parameters, and it must work with the `init_par` argument of `dapper_sample()`. The function returns an $n \times p$ numeric matrix of i.i.d. draws from $f(\cdot | \theta)$, with one proposed replacement for each record. During the subsequent sweep, row i is used as the proposal when updating x_i . Generating all proposals in one call also permits vectorized implementations. The matrix requirement is strict, so even if $p = 1$, `latent_f()` should return an $n \times 1$ matrix rather than a vector of length n .
- `mechanism_f()` returns the log density or log probability mass of the observed privatized statistic `sdp`, evaluated at `sx`. Terms that are constant with respect to `sx` may be omitted. Its syntax is `mechanism_f(sdp, sx)`, where `sdp` represents the observed privatized statistic and `sx` contains the summed record-level contributions. Both arguments must be numeric vectors or matrices and must appear in the stated order. The return value must be a scalar.
- `statistic_f()` returns the contribution of one latent record to the record-additive statistic. Its syntax must be `statistic_f(xi, sdp, i)`, where `xi` contains the i th latent record, `sdp` is the observed privatized statistic, and `i` is the record index. The function must return a numeric vector or matrix. The returned contributions must have compatible dimensions so that they can be summed, and their sum must have the form expected by `mechanism_f()`. In many applications, t_i depends only on `xi` and possibly `i`, so `sdp` is unused. The `sdp` argument supports more general record-additive statistics whose record-level contributions depend on the observed private release.
- `npar` is an integer representing the dimension of θ .
- `varnames` is an optional character vector containing the parameter names. These names are used to label the posterior draws and summary output. When supplied, it should contain one name for each of the `npar` parameters.

4.3 Sampling

The `dapper_sample()` function essentially takes an existing Bayesian model and extends it to handle privatized data. The output of `dapper_sample()` contains MCMC draws from the private posterior. The function has the following syntax:

```
dapper_sample(data_model = NULL, sdp = NULL, init_par = NULL, seed = NULL,
              niter = 2000, warmup = floor(niter / 2), chains = 1)
```

The parameters `data_model`, `sdp`, and `init_par` are required. The `data_model` input is a privacy object that is constructed using `new_privacy()` (see Section 4.2). The value of `sdp` is equal to the observed noise-infused statistic. We require the object class of `sdp` to be the same as the output of `statistic_f()`. For example, if `statistic_f()` returns a matrix, then `sdp` must also be a matrix. The provided starting value of the chain (`init_par`) must work with the `latent_f()` component. An error will be thrown if `latent_f()` evaluated at `init_par` does not return a numeric matrix.

The `niter` argument specifies the total number of MCMC iterations run per chain, including warmup, and `warmup` specifies how many initial iterations are discarded. Each chain therefore contributes `niter - warmup` retained post-warmup draws, and the returned chain contains `chains * (niter - warmup)` draws in total. The other optional arguments are `seed`, which sets the random seed, and `chains`, which specifies the number of chains. Running a chain without warmup can be done by setting `warmup = 0`. Running multiple chains can be done in parallel using the [furrr](#) (Vaughan et al., 2026) package. Additionally, progress can be monitored using the [progressr](#) (Bengtsson, 2026) package. Adhering to the design philosophy of the two packages, we leave the setup to the user so that they may choose the most appropriate configuration for their system. The contingency table demonstration given in Section 5 walks through a typical setup of [furrr](#) and [progressr](#).

The return value of `dapper_sample()` is a list containing a chain component stored as a `draws_matrix` object with `chains * (niter - warmup)` rows. The `mean_accept` component is a matrix with `niter - warmup` rows and `chains` columns. Each entry gives the fraction of latent record proposals accepted for the corresponding iteration and chain. The `comp_accept` component is an $n \times \text{chains}$ matrix giving the mean acceptance rate for each latent record over the post-warmup iterations. The `draws_matrix` object is described in more detail in the `posterior` (Bürkner et al., 2026) package. The advantages of working with a `draws_matrix` object are that it can be manipulated using `posterior` and is compatible with many packages in the `rstan` ecosystem. For example, a `draws_matrix` object can be passed directly to the popular `bayesplot` (Gabry and Mahr, 2026) package. Example 1 demonstrates both interfaces explicitly. Additionally, `dapper`'s basic summary function provides the same posterior summary statistics as those found when using `rstan`. Overall, this should make working with `dapper` easier for anyone already familiar with the `rstan` ecosystem.

4.4 Privacy mechanisms for count data

The `dapper` package provides several utility functions for analyzing privatized count data. Privatized count data are common in official and public statistics. As an example, the most prominent deployment of differential privacy for public data dissemination is the 2020 U.S. Decennial Census, consisting of a collection of tabular data products that follow a geographic hierarchy.

Pure ϵ -differential privacy places a stringent requirement on the privacy noise that must be introduced, which can lead to poor data quality. For this reason, alternative choices of privacy noise can be desirable. For example, the U.S. Census Bureau adopts zero-concentrated differential privacy (zCDP), a more general criterion that permits a broader range of noise mechanisms. The `dapper` package includes probability mass and sampling functions for the discrete Gaussian and discrete Laplace distributions (Canonne et al., 2022), both of which are commonly used in the zCDP framework. The 2020 U.S. Decennial Census data products are protected with two sets of mechanisms that satisfy zCDP: the TopDown mechanism for the redistricting and Demographic and Housing Characteristics (DHC) files (Abowd et al., 2022), and the SafeTab mechanism for the detailed DHC files (Tumult Labs, 2022).

Equations (1) and (2) in the panel below give the probability mass functions for the discrete Gaussian and discrete Laplace distributions, respectively:

$$P[X = x] = \frac{e^{-(x-\mu)^2/(2\sigma^2)}}{\sum_{y \in \mathbb{Z}} e^{-(y-\mu)^2/(2\sigma^2)}}, \quad (1)$$

$$P[X = x] = \frac{e^{1/t} - 1}{e^{1/t} + 1} e^{-|x|/t}. \quad (2)$$

The support of both distributions is the set of all integers. The discrete Gaussian has two parameters $(\mu, \sigma) \in \mathbb{R} \times \mathbb{R}^+$, which govern the location and scale, respectively. On the other hand, the discrete Laplace only has the scale parameter $t \in \mathbb{R}^+$. The functions `ddnorm()` and `rdnorm()` provide the density and sampling features for the discrete Gaussian distribution. The normalizing constant used by `ddnorm()` is cached for reuse across calls with the same sigma. When `log = TRUE`, the function returns unnormalized log probabilities. Finally, the functions `ddlplace()` and `rdlplace()` provide similar features for the discrete Laplace distribution. The functions `ddnorm()` and `rdnorm()` require an integer `mu`. For the discrete Laplace distribution, `ddlplace()` accepts any positive scale, while `rdlplace()` requires a positive integer scale. Due to potential floating-point attacks (Mironov, 2012), the implementation of these functions is not the safest and is merely meant to provide users with a way to quickly explore the behavior of these DP mechanisms. The implementation logic for both the discrete Gaussian and discrete Laplace mechanisms closely follows the algorithms as outlined in Canonne et al. (2022).

5 Examples

5.1 Example 1: 2×2 contingency table (randomized response)

As a demonstration, we analyze a subset of the UC Berkeley admissions data, which is often used as an illustrative example of Simpson's paradox (Bickel et al., 1975). The question posed is whether the data suggest there is bias against females during the college admissions process. We work with a $\sim 9\%$ subsample consisting of $N = 400$ applicants for computational convenience. The paired tables below show admissions outcomes aggregated across six departments by sex for this subsample. The left

Table 2: The left-hand table shows the confidential admissions data, and the right-hand table shows the perturbed counts after application of the randomized response mechanism. The mechanism substantially reduces the observed difference in admission rates between female and male applicants.

	Admitted	Rejected		Admitted	Rejected
Female	46	118	Female	74	102
Male	109	127	Male	104	120

subtable shows the confidential data, and the right subtable shows the resulting counts after applying the randomized response privacy mechanism.

To see how the privacy mechanism works, we represent the record-level data as an $N \times 2$ matrix, with the first column indicating sex and the second indicating admission status. Each row therefore represents one individual. We apply randomized response independently to each binary attribute: the original value is preserved with probability $3/4$ and flipped with probability $1/4$.² This rule can, for example, be implemented by flipping two fair coins for each attribute and changing the value only for one designated combination of outcomes. Thus, for an individual recorded as male and rejected, the sex and admission-status values are randomized separately. Applying the mechanism to both attributes gives a privacy loss budget of at most $\epsilon = 2 \log(3)$.

To set up `dapper` to analyze the anonymized admissions data, we encode the randomized record-level data as a length-800 vector. This vector is formed by stacking the sex column of the binary data matrix above the admission-status column, where male and admitted take the value 1.

1. `latent_f`: For each individual there are four possible sex/status responses, which can be modeled using a multinomial distribution. To implement draws from the multinomial distribution, we use the `sample()` function to take samples from a list containing the four possible binary vectors. Note that the final line results in a 400×2 matrix.

```
latent_f <- function(theta) {
  t1 <- list(c(1,1), c(1,0), c(0,1), c(0,0))
  rs <- sample(t1, 400, replace = TRUE, prob = theta)
  do.call(rbind, rs)
}
```

2. `posterior_f`: Given the confidential data, we can derive the posterior analytically using a Dirichlet prior. In this example, we use a flat prior, which corresponds to a Dirichlet(1) distribution. The code below generates samples from the Dirichlet distribution using random draws from the gamma distribution.

```
posterior_f <- function(dmat, theta) {
  sex <- dmat[,1]
  status <- dmat[,2]

  #Male & Admit
  x1 <- sum(sex & status)
  x2 <- sum(sex & !status)
  x3 <- sum(!sex & status)
  x4 <- sum(!sex & !status)

  x <- c(x1, x2, x3, x4)

  t1 <- rgamma(4, x + 1, 1)
  t1/sum(t1)
}
```

3. `statistic_f`: Because randomized response is applied independently to the two attributes, the mechanism likelihood depends on how many privatized attributes match the corresponding latent attributes. For record i , define

$$t_i(x_i, s_{dp}) = \mathbb{1}(x_{i1} = s_{dp,i}) + \mathbb{1}(x_{i2} = s_{dp,n+i}).$$

Summing these contributions gives the total number of matching entries across the database.

²The randomized response scheme predates the development of differential privacy and was first described by Warner (1965) as a means to reduce survey bias involving sensitive questions.

```

statistic_f <- function(xi, sdp, i) {
  n <- length(sdp) %% 2
  sum(xi == sdp[c(i, n + i)])
}

```

4. `mechanism_f`: Each privatized attribute matches its confidential value with probability $3/4$ and differs with probability $1/4$. If sx is the total number of matching entries, the log likelihood is

$$sx \log(3/4) + (2n - sx) \log(1/4).$$

```

mechanism_f <- function(sdp, sx) {
  sx * log(3/4) + (length(sdp) - sx) * log(1/4)
}

```

Below, we load the data and create the noisy admissions table.

```

#Original UCBA admissions data.
cnf_df <- tibble(sex = c(1, 1, 0, 0),
  status = c(1, 0, 1, 0),
  n = c(1198, 1493, 557, 1278)) %>% uncount(n)

set.seed(1)
ix <- sample(1:nrow(cnf_df), 400, replace = FALSE)
cnf_df <- cnf_df[ix,]

#Answers to be randomized
ri <- as.logical(rbinom(800, 1, 1/2))

#Randomized answers
ra <- rbinom(sum(ri), 1, 1/2)

#Create sdp
sdp <- c(as.matrix(cnf_df))
sdp[ri] <- ra

```

Once we have defined all components of the model, we can create a new privacy model object using the `new_privacy()` function and feed this into the `dapper_sample()` function. Below, we run four chains in parallel for 6,000 total MCMC iterations per chain, discarding the first 1,000 iterations as warmup. This leaves 5,000 retained post-warmup draws per chain and 20,000 pooled draws across the four chains.³

```

library(dapper)
library(furrr)
plan(multisession, workers = 4)

dmod <- new_privacy(posterior_f = posterior_f,
  latent_f = latent_f,
  mechanism_f = mechanism_f,
  statistic_f = statistic_f,
  npar = 4,
  varnames = c("pi_11", "pi_10", "pi_01", "pi_00"))

dp_out <- dapper_sample(dmod,
  sdp = sdp,
  seed = 123,
  niter = 6000,
  warmup = 1000,
  chains = 4,
  init_par = rep(.25, 4))

```

If the runtime of `dapper_sample()` is exceptionally long, one can use the [progressr](#) package to monitor progress. The [progressr](#) framework allows for a unified handling of progress bars in both the sequential and parallel computing cases.

³The number of retained post-warmup draws used in each example is chosen for computational convenience, ensuring a reasonable effective sample size within a short runtime. In a formal analysis, however, a larger effective sample size is typically required, particularly when estimating posterior quantiles near the tails.

```
library(progressr)
handlers(global = TRUE)

handlers("cli")
dp_out <- dapper_sample(dmod,
  sdp = sdp,
  seed = 123,
  niter = 6000,
  warmup = 1000,
  chains = 4,
  init_par = rep(.25,4))
```

Results can be quickly summarized using `summary()`, which delegates to `summarise_draws()` from the **posterior** package. The `rhat` values in the table below are close to 1, which is consistent with convergence. They should be considered together with the trace plots and effective sample sizes.

```
#> # A tibble: 4 x 10
#>   variable mean median    sd    mad    q5    q95  rhat ess_bulk ess_tail
#>   <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>    <dbl>
#> 1 pi_11    0.281  0.281 0.0610 0.0625 0.182  0.382  1.02    362.    818.
#> 2 pi_10    0.336  0.335 0.0638 0.0640 0.235  0.444  1.01    431.   1191.
#> 3 pi_01    0.111  0.108 0.0548 0.0563 0.0250 0.206  1.02    282.    504.
#> 4 pi_00    0.272  0.272 0.0601 0.0616 0.172  0.372  1.02    389.    875.
```

Because `dp_out$chain` is a `draws_matrix`, it can be passed directly to **bayesplot**. The call underlying `plot(dp_out)` is shown explicitly in Figure 2. It is especially important to check for good mixing with **dapper** since sticky chains are likely to be produced when the amount of injected noise is high. See Section 6.2 for a more detailed explanation.

```
bayesplot::mcmc_trace(dp_out$chain) +
  ggsci::scale_color_lancet() +
  paper_theme +
  guides(color = guide_legend(title = "Chain", title.position = "left", nrow = 1))
```

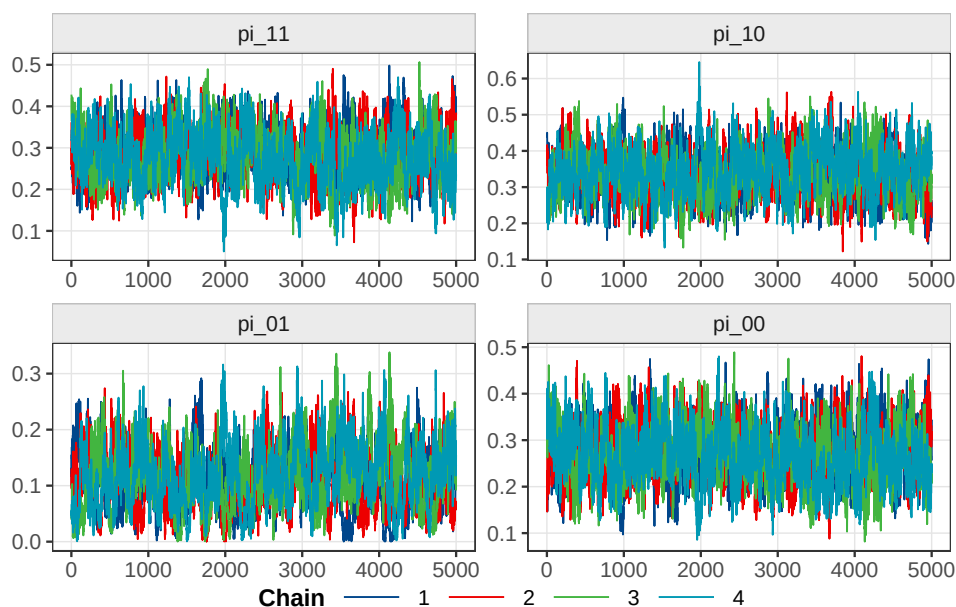


Figure 2: (Example 1) Trace plots created by passing the **dapper** draws directly to **bayesplot**. Each chain is represented by a distinct qualitative color. By returning posterior draws in a standard format, **dapper** makes it easy to use existing Bayesian diagnostic tools such as **bayesplot**.

To see if there is evidence of gender bias, we can look at the odds ratio. Specifically, we compare the odds of a male applicant being admitted with those of a female applicant. A higher odds ratio would indicate a bias favoring males. The `mutate_variables()` and `subset_draws()` functions from **posterior** can be used to create and select this derived posterior quantity without converting the **dapper** output to another format.

```
odds_ratio_draws <- posterior::mutate_variables(
  dp_out$chain,
  odds_ratio = (pi_11 * pi_00) / (pi_10 * pi_01)
) |>
  posterior::subset_draws(variable = "odds_ratio")

posterior::summarise_draws(odds_ratio_draws)

#> # A tibble: 1 x 10
#>   variable    mean median    sd  mad    q5   q95  rhat ess_bulk ess_tail
#>   <chr>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>    <dbl>    <dbl>
#> 1 odds_ratio  6.83   2.12 82.0  1.89 0.472 16.7  1.03     245.     568.
```

Figure 3 visualizes the derived odds ratio using **bayesplot**. The large odds ratio values would seem to indicate that there is bias favoring males. Stratifying the data by university department reveals Simpson's paradox. The within-department comparisons reverse the aggregate pattern and tend to favor female applicants.

```
previous_color_scheme <- attr(bayesplot::color_scheme_get(), "scheme_name")
bayesplot::color_scheme_set(privatized_color_scheme)

odds_ratio_plot <- bayesplot::mcmc_areas(
  odds_ratio_draws,
  prob = 0.8,
  prob_outer = 0.95,
  point_est = "none"
) +
  geom_vline(xintercept = 1, linetype = "dashed", linewidth = 0.5) +
  coord_cartesian(xlim = c(0, 10)) +
  labs(x = "Odds Ratio", y = NULL) +
  paper_theme +
  theme(
    axis.text.y = element_blank(),
    axis.ticks.y = element_blank()
  )

bayesplot::color_scheme_set(previous_color_scheme)
odds_ratio_plot
```

alt=Red density estimate of the randomized-response odds ratio, with a central 80 percent interval and a vertical reference line at one.

Figure 3: (Example 1) Posterior density estimate for the odds ratio, displayed over the range 0 to 10 and based on 20,000 retained post-warmup draws (5,000 from each of four chains). The light-red region denotes the central 80% interval, and the dashed vertical line marks the null value of 1. The posterior is strongly right-skewed, and its central 80% interval includes the null value of 1.

For comparison, we run a standard Bayesian analysis on the noise-infused table ignoring the privacy mechanism. This will correspond exactly to the model defined in the `posterior_f` component. Figure 4 shows a density estimate for the odds ratio under the confidential and noisy data. The posterior distribution for the odds ratio under the noisy data is shifted significantly, indicating a large degree of bias. The left-hand plot in Figure 4 shows that the MAP estimate from **dapper** is similar to that in the case of the confidential data. The width of the posterior is also much larger since it properly accounts for the uncertainty due to the privacy mechanism. This illustrates the dangers of ignoring the privacy mechanism: a naïve analysis is not only biased but also severely underestimates the uncertainty associated with the odds ratio estimate.

Table 3: The left-hand table shows the confidential admissions data, and the right-hand table shows the perturbed data resulting from the discrete Gaussian mechanism with the scale parameter set to 6.32. The perturbed counts remain close to the confidential counts and retain the higher observed admission rate for male applicants.

	Admitted	Rejected		Admitted	Rejected
Female	46	118	Female	47	110
Male	109	127	Male	110	131

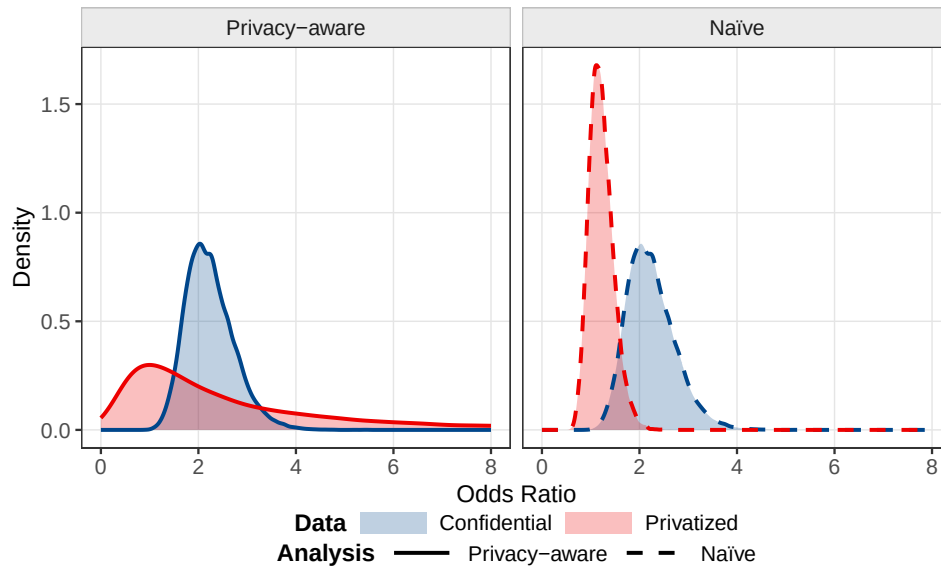


Figure 4: (Example 1) Posterior distributions of the odds ratio (admission of males vs. females). Blue denotes confidential data and red denotes privatized data. Solid lines denote privacy-aware inference using dapper; dashed lines denote naïve inference that treats the privatized data as noise-free. Accounting for randomized response substantially increases posterior uncertainty relative to the naïve analysis.

5.2 Example 2: 2×2 contingency table (discrete Gaussian)

To highlight the flexibility of **dapper**, we reanalyze Example 1 with a different privacy mechanism. Here we take inspiration from the 2020 U.S. Decennial Census, which deployed a novel privacy protection system for count data. In particular, the discrete Gaussian distribution is used in SafeTab. In our admissions data example, this privacy mechanism works by injecting noise into the total cell counts given in the 2×2 table. The randomized response scheme, in contrast, injects noise at the record level. The **dapper** package accommodates both the randomized response and the discrete Gaussian mechanisms, allowing us to compare the impact of the two approaches.

These mechanisms also illustrate two different trust models. Randomized response is a local differential privacy mechanism in which each individual's attributes are privatized before being made available to the data curator. On the other hand, the discrete Gaussian mechanism operates under central differential privacy, in which a trusted curator collects the confidential records and adds noise to the aggregate counts before release. Local differential privacy therefore protects the data from the curator as well as downstream users, but this stronger protection can entail substantially greater statistical cost for many estimation problems (Duchi et al., 2018; Barber and Duchi, 2014).

To begin comparing the two approaches, we need to set the privacy parameters. Since the two frameworks use different metrics, there does not exist a direct comparison. However, there is a direct relationship between ϵ -CDP and (ϵ, δ) -differential privacy. The latter framework is a relaxed version of ϵ -DP where the ratio bound only holds in probability. For our comparison, we will set $\delta = 10^{-10}$, which is the value used in the 2020 Decennial Census. For this value of δ , setting the scale parameter in the discrete Gaussian distribution to $\sigma = 6.32$ will guarantee $(2 \log(3), 10^{-10})$ -differential privacy.⁴

⁴ $\frac{1}{2}\epsilon^2$ -concentrated differential privacy implies $(\frac{1}{2}\epsilon^2 + \epsilon\sqrt{2\log(1/\delta)}, \delta)$ -differential privacy (Bun and Steinke, 2016).

For the public, the 2020 Decennial Census only shows aggregate cell counts along with the true total count of those cells. In our example, this means the Census would have released the right-hand table along with the fact that $N = 400$ in the original table. Thus, it is natural to let s_{dp} be the vector of cell counts. As in Example 1, we imagine the latent database as a 400×2 binary matrix. Below, we describe the process for analyzing the privatized data using **dapper**. Since the latent process and posterior are the same as in Example 1, we only describe how to construct `statistic_f` and `mechanism_f`.

1. `statistic_f`: The private summary statistic s_{dp} can be written as a record-additive statistic using the indicator vectors $(1, 0, 0, 0)$, $(0, 1, 0, 0)$, $(0, 0, 1, 0)$, and $(0, 0, 0, 1)$. These four vectors correspond to the four possible cells.

```
statistic_f <- function(xi, sdp, i) {
  if(xi[1] & xi[2]) {
    c(1, 0, 0, 0)
  } else if (xi[1] & !xi[2]) {
    c(0, 1, 0, 0)
  } else if (!xi[1] & xi[2]) {
    c(0, 0, 1, 0)
  } else {
    c(0, 0, 0, 1)
  }
}
```

2. `mechanism_f`: The privacy mechanism is a discrete Gaussian distribution centered at 0.

```
mechanism_f <- function(sdp, sx) {
  sum(dapper::ddnorm(sdp - sx, mu = 0, sigma = 6.32, log = TRUE))
}
```

The summary table below shows the results from one chain run for 2,000 total MCMC iterations. The first 1,000 iterations are discarded as warmup, leaving 1,000 retained post-warmup draws.

```
summary(dp_out)
```

```
#> # A tibble: 4 x 10
#>   variable mean median    sd    mad    q5    q95  rhat ess_bulk ess_tail
#>   <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>    <dbl>
#> 1 pi_11    0.278  0.277 0.0247 0.0234 0.238  0.321 1.00      624.    778.
#> 2 pi_10    0.326  0.325 0.0269 0.0263 0.282  0.372 1.00      586.    546.
#> 3 pi_01    0.120  0.120 0.0213 0.0214 0.0855 0.156 1.000     450.    670.
#> 4 pi_00    0.276  0.276 0.0264 0.0266 0.234  0.321 1.00      644.    680.
```

Figure 5 juxtaposes the private posterior under the randomized response (dashed line) and discrete Gaussian (solid line) mechanisms. The comparison suggests that using discrete Gaussian noise leads to slightly less posterior uncertainty and a mode closer to that of the confidential-data posterior.

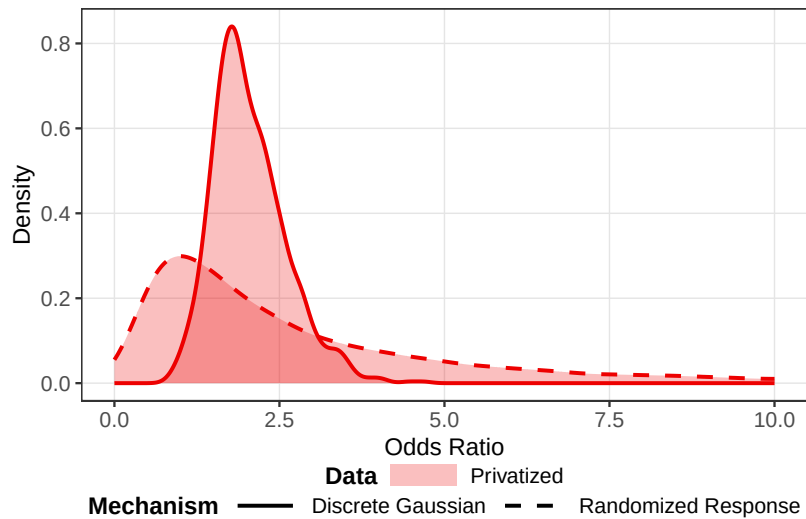


Figure 5: (Example 2) Privacy-aware posterior density estimates for the odds ratio based on privatized data (red). Dashed and solid lines denote the randomized response and discrete Gaussian mechanisms, respectively. The estimates are based on 20,000 retained post-warmup draws for randomized response (5,000 from each of four chains) and 1,000 retained post-warmup draws from one chain for the discrete Gaussian mechanism. For these releases, the discrete Gaussian mechanism yields a more concentrated odds-ratio posterior than randomized response.

Additionally, comparing the plots in Figures 4 and 6 shows that the discrete Gaussian mechanism induces considerably less bias under the naïve analysis.

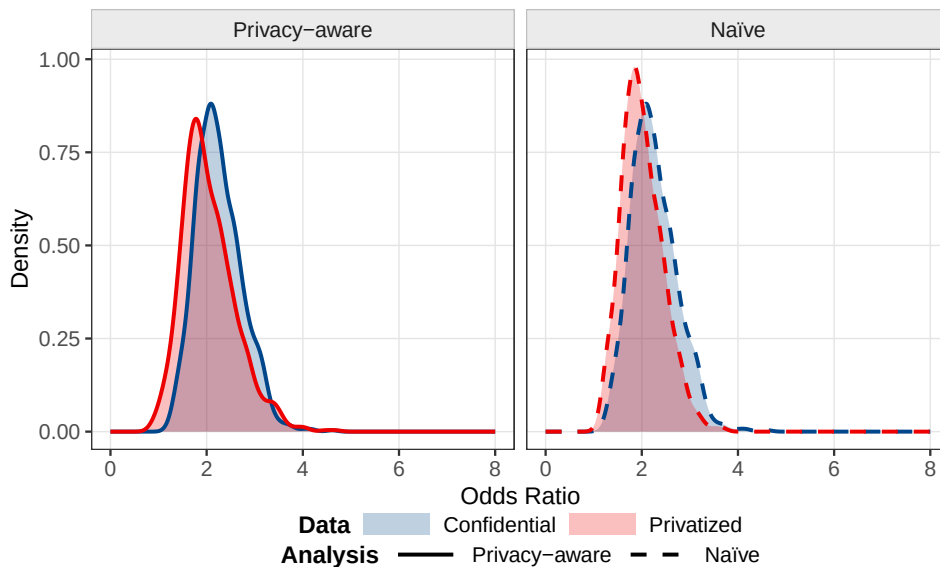


Figure 6: (Example 2) Posterior distributions of the odds ratio (admission of males vs. females). Blue denotes confidential data and red denotes privatized data. Solid lines denote privacy-aware inference using dapper; dashed lines denote naïve inference that treats the privatized data as noise-free. The privacy-aware posterior closely resembles the posterior obtained from the confidential data.

5.3 Example 3: linear regression

In this section we apply [dapper](#) to reconstruct an example presented in [Ju et al. \(2022\)](#). In it, they apply a Laplace privacy mechanism to a sufficient summary statistic for a linear regression model. Let $\{(x_i, y_i)\}_{i=1}^n$ be the original, confidential data with $x_i \in \mathbb{R}^2$. They assume the true data-generating process follows the model

$$\begin{aligned}
y &= -1.79 - 2.89x_1 - 0.66x_2 + \epsilon \\
\epsilon &\sim N(0, \sigma^2), \quad \sigma^2 = 2 \\
\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} &\sim N_2(\mu, I_2) \\
\mu &= \begin{pmatrix} 0.9 \\ -1.17 \end{pmatrix}.
\end{aligned}$$

In most settings involving linear regression, the covariates are assumed to be fixed, known constants. Thus, the formulation above is a departure from the norm since we are assuming a random design matrix. More details on why this framing is necessary will be provided later when describing the latent model. The paper considers the scenario where one desires to publicly release the sufficient summary statistics

$$s(x, y) = (x^T y, y^T y, x^T x).$$

This summary statistic satisfies the record-additive property since $s(x, y) = \sum_{i=1}^n t(x_i, y_i)$, where

$$t(x_i, y_i) = ((x_i)^T y_i, y_i^2, (x_i)^T x_i).$$

To make the statistic compliant with the ϵ -DP criterion, it is necessary to bound the value of the statistic (i.e., the statistic must have finite global sensitivity). This is accomplished by clamping the data, which prevents any data point from being too “unique.” More precisely, we define the clamping function $[z] := \min\{\max\{z, -10\}, 10\}$ which truncates a value z so that it falls into the interval $[-10, 10]$. Furthermore, we let $\tilde{z} := [z]/10$ denote the normalized clamped value of z . The clamped statistic is

$$t(x_i, y_i) = ((\tilde{x}_i)^T \tilde{y}_i, \tilde{y}_i^2, (\tilde{x}_i)^T \tilde{x}_i).$$

Ignoring duplicate entries in the symmetric matrix $x^T x$, the statistic has ℓ_1 -sensitivity $\Delta = p^2 + 4p + 3$, where p is the number of predictors in the regression model (in this example $p = 2$).⁵ Using the Laplace mechanism, ϵ -DP can thus be achieved by adding i.i.d. $\text{Laplace}(0, \Delta/\epsilon)$ error to each unique entry. For techniques that yield a tighter sensitivity bound, see [Awan and Slavković \(2021\)](#).

1. `latent_f`: Because the summary statistic encodes information about the design matrix and the privacy mechanism adds noise to that statistic, we cannot treat the design matrix as fixed and known. Hence, to draw a sample from the latent data-generating process, we use the relation $f(x, y) = f(x)f(y | x)$. In this formulation, it is necessary to specify a distribution on the covariates x .

```
latent_f <- function(theta) {
  xmat <- MASS::mvrnorm(50, mu = c(.9, -1.17), Sigma = diag(2))
  y <- cbind(1, xmat) %*% theta + rnorm(50, sd = sqrt(2))
  cbind(y, xmat)
}
```

2. `posterior_f`: Given confidential data x , we can derive the posterior analytically using a normal prior on β .

$$\begin{aligned}
\beta &\sim N_{p+1}(0, \tau^2 I_{p+1}) \\
\beta | x, y &\sim N(\mu_n, \Sigma_n) \\
\Sigma_n &= (x^T x / \sigma^2 + I_{p+1} / \tau^2)^{-1} \\
\mu_n &= \Sigma_n (x^T y) / \sigma^2
\end{aligned}$$

In the example, we use $\sigma^2 = 2$ and $\tau^2 = 4$.

```
posterior_f <- function(dmat, theta) {
  x <- cbind(1, dmat[, -1])
  y <- dmat[, 1]

  ps_s2 <- solve((1/2) * t(x) %*% x + (1/4) * diag(3))
  ps_m <- ps_s2 %*% (t(x) %*% y) * (1/2)

  MASS::mvrnorm(1, mu = ps_m, Sigma = ps_s2)
}
```

⁵The original paper, [Ju et al. \(2022\)](#), contains a computation error and mistakenly uses $\Delta = p^2 + 3p + 3$.

3. `statistic_f`: The summary statistic contains duplicate entries. We can considerably reduce the dimension of the statistic by only considering unique entries. The `clamp_data()` function is used to bound the statistic to give a finite global sensitivity.

```
clamp_data <- function(dmat) {
  pmin(pmax(dmat,-10),10) / 10
}

statistic_f <- function(xi, sdp, i) {
  xic <- clamp_data(xi)
  ydp <- xic[1]
  xdp <- cbind(1,t(xic[-1]))

  s1 <- t(xdp) %*% ydp
  s2 <- t(ydp) %*% ydp
  s3 <- t(xdp) %*% xdp

  ur_s1 <- c(s1)
  ur_s2 <- c(s2)
  ur_s3 <- s3[upper.tri(s3,diag = TRUE)][-1]
  c(ur_s1,ur_s2,ur_s3)
}
```

4. `mechanism_f`: The privacy mechanism adds $\text{Laplace}(0, \Delta/\epsilon)$ error to each unique entry of the statistic. In this example, $\Delta = 15$ and $\epsilon = 10$.

```
mechanism_f <- function(sdp, sx) {
  sum(VGAM::dlaplace(sdp - sx, 0, 15/10, log = TRUE))
}
```

First, we simulate fake data using the aforementioned privacy mechanism. In the example, we use $n = 50$ observations.

```
deltaa <- 15
epsilon <- 10
n <- 50

set.seed(1)
xmat <- MASS::mvrnorm(n, mu = c(.9,-1.17), Sigma = diag(2))
beta <- c(-1.79, -2.89, -0.66)
y <- cbind(1,xmat) %*% beta + rnorm(n, sd = sqrt(2))

#clamp the confidential data in xmat
dmat <- cbind(y,xmat)
sdp <- apply(
  sapply(seq_len(nrow(dmat)),
    function(i) {
      statistic_f(dmat[i, ], NULL, i)
    }),
  1,
  sum
)

#add Laplace noise
sdp <- sdp + VGAM::rlaplace(length(sdp), location = 0, scale = deltaa/epsilon)
```

We construct a privacy model using the `new_privacy()` function and run one chain for 25,000 total MCMC iterations. The first 1,000 iterations are discarded as warmup, leaving 24,000 retained post-warmup draws.

```
library(dapper)
set.seed(123)

dmod <- new_privacy(posterior_f = posterior_f,
  latent_f = latent_f,
```

```

mechanism_f = mechanism_f,
statistic_f = statistic_f,
npar       = 3,
varnames   = c("beta0", "beta1", "beta2"))

dp_out <- dapper_sample(dmod,
  sdp = sdp,
  niter = 25000,
  warmup = 1000,
  chains = 1,
  init_par = rep(0,3))

```

The output of the MCMC run is reported below.

```

summary(dp_out)

#> # A tibble: 3 x 10
#>   variable mean median   sd mad   q5   q95 rhat ess_bulk ess_tail
#>   <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>    <dbl>    <dbl>
#> 1 beta0   -0.978 -0.894  1.53  1.52 -3.64  1.44  1.01    437.    731.
#> 2 beta1   -1.86  -2.23  1.53  1.20 -3.73  1.39  1.00    127.    192.
#> 3 beta2    0.729  0.659  1.40  1.57 -1.47  3.02  1.00    145.    524.

```

For comparison, we consider a Bayesian analysis in which the design matrix is a fixed, known constant and σ^2 is known. This fixed-design analysis does not exactly correspond to the model used in `posterior_f()`, but it serves as the naïve analysis. However, under a prior in which the regression and design-matrix parameters are independent, the design-matrix parameters can be profiled out. As such, inference for β can be carried out by conditioning on the observed covariates. Using the diffuse prior $f(\beta) \propto 1$ leads to a normal posterior.

$$\begin{aligned}
 \beta \mid x, y, \sigma^2 &\sim N(\hat{\beta}, \hat{\Sigma}) \\
 \hat{\beta} &= (x^T x)^{-1} x^T y \\
 \hat{\Sigma} &= \sigma^2 (x^T x)^{-1}.
 \end{aligned}$$

The posterior can be written as a function of $s(x, y)$, but since we only have access to the noisy version s_{dp} , we attempt to reconstruct it by extracting the relevant entries, as shown below.

```

#x^Ty
s1 <- sdp[1:3]

#y^Ty
s2 <- sdp[4]

#x^Tx
s3 <- matrix(0, nrow = 3, ncol = 3)
s3[upper.tri(s3, diag = TRUE)] <- c(n, sdp[5:9])
s3[lower.tri(s3)] <- s3[upper.tri(s3)]

```

Because of the injected privacy noise, the reconstructed $(x^T x)^{-1}$ matrix is not positive definite. As a naïve solution, we use `nearest_spd()` from the `pracma` package (Borchers, 2025), which implements the algorithm proposed by Higham (1988) and produces a positive-definite approximation under the Frobenius norm.

The released sufficient statistics are calculated after dividing both the response and predictors by 10. Apart from values affected by clamping, the corresponding regression therefore has coefficient vector $(\beta_0/10, \beta_1, \beta_2)$ and residual variance $\sigma^2/10^2$. To report the naïve posterior on the original parameter scale, we transform its mean and covariance using $A = \text{diag}(10, 1, 1)$.

```

s3_inverse <- pracma::nearest_spd(solve(s3))
parameter_scale <- diag(c(10, 1, 1))

```

```

bhat_scaled <- s3_inverse %*% s1
sigma_hat_scaled <- (2 / 10^2) * s3_inverse

bhat <- parameter_scale %*% bhat_scaled
sigma_hat <- parameter_scale %*% sigma_hat_scaled %*% parameter_scale

```

Figure 7 shows the posterior density estimates for the β coefficients based on s_{dp} . The density estimates indicate that the naïve method, which ignores the privacy mechanism, is biased and underestimates the variance. The especially narrow naïve densities are not a rescaling artifact. In this simulated release, privacy noise makes the reconstructed $x^T x$ matrix indefinite. The positive-definite approximation to its inverse nearly collapses one covariance direction, producing spuriously high precision when the release is treated as noise-free. Applying the same rescaling to the unnoised statistics recovers the confidential-data covariance in this example. Likewise, Figure 8 illustrates how **dapper** provides point estimates that are not far off from those that would have been obtained using the original confidential data. The dramatic increase in the posterior variance indicates the privacy mechanism adds substantial uncertainty to the estimates.

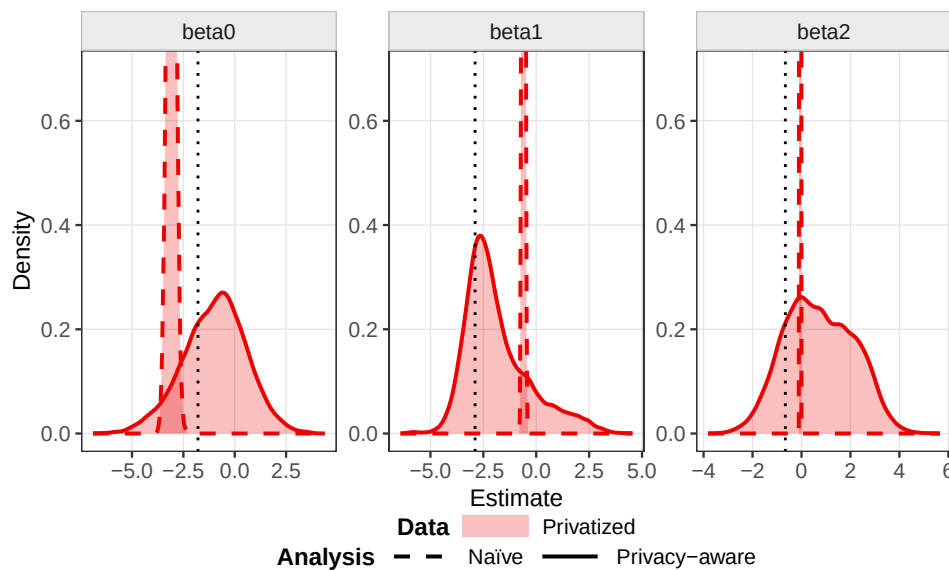


Figure 7: (Example 3) Regression-coefficient posteriors based on privatized data (red). Solid lines denote privacy-aware inference using **dapper** and dashed lines denote naïve inference that treats the privatized data as noise-free. Dotted vertical lines mark the true coefficient values. Treating the privatized statistics as noise-free produces extremely narrow posteriors displaced from the true coefficient values.

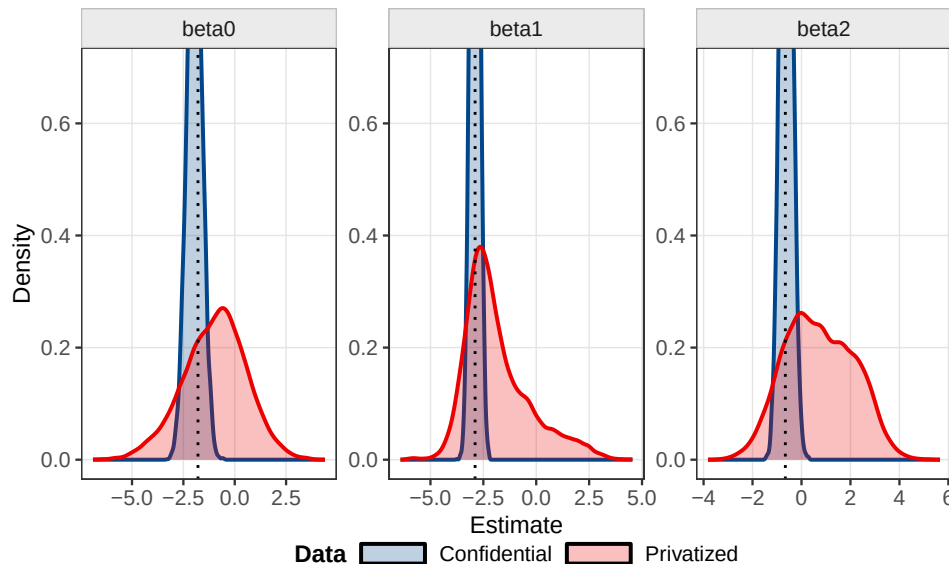


Figure 8: (Example 3) Privacy-aware regression-coefficient posteriors using dapper. Blue denotes confidential data and red denotes privatized data. Dotted vertical lines mark the true coefficient values. Accounting for privacy noise yields much broader coefficient posteriors than the confidential-data analysis, reflecting the information lost through privatization.

Posterior predictive checks

Posterior predictive checks assess whether data generated under the fitted model resemble the observed data in features relevant to the analysis. In this setting, the observed data are the privatized sufficient statistics rather than the confidential records. We therefore reproduce the full data-release process for each posterior draw: generate a replicated confidential data set with `latent_f()`, calculate its clamped sufficient statistic with `statistic_f()`, and add fresh noise from the same Laplace mechanism. This produces draws from the posterior predictive distribution of the private release.

Because s_{dp} is multivariate, we illustrate the procedure using its $y^T y$ component, which measures the overall magnitude of the response. For every replicated data set, the fourth component returned by `statistic_f()` is the clamped response sum of squares. Figure 9 compares the observed private value with 500 replicated private values using the `ppc_stat()` function from `bayesplot`.

```
set.seed(123)
ppc_draw_ids <- sample(seq_len(nrow(dp_out$chain)), size = 500)

replicated_private_yty <- vapply(ppc_draw_ids, function(draw_id) {
  simulate_private_yty(as.numeric(dp_out$chain[draw_id, ]))
}, numeric(1))

bayesplot::ppc_stat(
  y = sdp[4],
  yrep = matrix(replicated_private_yty, ncol = 1),
  stat = function(z) z[1],
  bins = 30,
  freq = FALSE
) +
  labs(x = expression("Privatized " * y^T * y), y = "Density") +
  paper_theme +
  bayesplot::legend_none()
```

alt=Histogram of 500 replicated privatized response sums of squares, with the observed private release marked by a red vertical line.

Figure 9: (Example 3) Posterior predictive distribution of the privatized response sum of squares. The red vertical line marks the observed private release, and the histogram shows 500 replicated private releases generated under the fitted model. This example demonstrates how dapper can be combined with existing tools such as bayesplot to perform posterior predictive checks on privatized data.

This check is limited to one component to demonstrate the feasibility of posterior predictive checking and is not meant to be a comprehensive assessment of the multivariate release. A more thorough check can be carried out by applying the same procedure to the components of $x^T y$ and $x^T x$, or extended using a joint discrepancy measure. Additionally, prior predictive checks are available by replacing the posterior draws with draws from the prior, while convergence and posterior diagnostics can be performed with [posterior](#) and [bayesplot](#).

Other parts of the Bayesian workflow can also be easily incorporated. For instance, simulation-based calibration may be useful for detecting implementation errors in this setting. One can repeatedly draw parameters from the prior, generate confidential data and a private release using the user-supplied components, fit the model with `dapper_sample()`, and check whether the generating parameter values have uniform ranks among the corresponding posterior draws. Systematic departures from uniformity can reveal inconsistencies among the latent model, statistic, privacy mechanism, and sampler. Checks requiring an unreleased feature of the confidential data would require an additional privacy-preserving release.

5.4 Example 4: wrapper-based sampler

In all the examples so far, we have used a conjugate prior for the model of the confidential data. However, many models in practice are more complex and require MCMC methods to draw samples. In this example, we illustrate how to use [dapper](#) with a non-conjugate prior by modifying the previous linear regression example to use the t -distribution as a prior for the regression coefficients. For this model, we first use the [fmcmm](#) package to construct a sampler for the latent model and then use [dapper](#) as a wrapper around the [fmcmm](#) sampler. Because [dapper](#) is modular, the only thing we need to change is the `posterior_f()` function. First, we construct a posterior sampler for the latent data model.⁶

```
library(fmcmm)

log_latent_posterior <- function(theta, x, y) {
  #t-distribution as prior
  log_prior_theta <- sum(dt(theta, df = 5, log = TRUE))

  mu_y <- cbind(1, x) %*% theta
  log_lik_y <- sum(dnorm(y, mean = mu_y, sd = sqrt(2), log = TRUE))

  log_prior_theta + log_lik_y
}
```

The [fmcmm](#) package comes with several proposal distributions. Here we choose the normal distribution, and some testing shows that setting the scale parameter to 1/5 works well. The sampler returns a [coda](#) (Plummer et al., 2006) object that works similarly to objects from the [posterior](#) package. Below is a summary of 2,000 retained post-burn-in draws from our previous naïve analysis using a t -distribution with 5 degrees of freedom as the prior for the regression coefficients. The sampler performs 3,000 total steps and discards the first 1,000 as burn-in.

```
latent_posterior <- MCMC(
  initial = rep(0, 3),
  fun     = log_latent_posterior,
  progress = FALSE,
  nsteps  = 3000,
  burnin  = 1000,
  kernel  = kernel_normal(scale = 0.2),
  x = dmat[, -1],
  y = dmat[, 1]
)

summary(latent_posterior)

#>
#> Iterations = 1001:3000
```

⁶The prior t -distribution parameters used in this example are for illustrative purposes. For a comprehensive discussion of how to construct t -distribution priors for generalized linear models, see [Gelman et al. \(2008\)](#).

```
#> Thinning interval = 1
#> Number of chains = 1
#> Sample size per chain = 2000
#>
#> 1. Empirical mean and standard deviation for each variable,
#>   plus standard error of the mean:
#>
#>      Mean      SD Naive SE Time-series SE
#> par1 -1.849 0.3951 0.008834      0.05980
#> par2 -2.893 0.2125 0.004753      0.01894
#> par3 -0.528 0.2399 0.005363      0.02909
#>
#> 2. Quantiles for each variable:
#>
#>      2.5%      25%      50%      75%      97.5%
#> par1 -2.5508 -2.1163 -1.8503 -1.5843 -0.95429
#> par2 -3.2888 -3.0459 -2.8938 -2.7468 -2.44055
#> par3 -0.9773 -0.6817 -0.5338 -0.3822 -0.01746
```

Next, we use **dapper** as a wrapper around this sampler to account for the privacy mechanism. Per-call burn-in can be expensive because it is repeated at every outer iteration, so it should generally be omitted or kept short when the wrapped sampler permits. Some adaptive samplers may nevertheless require a short initialization period. A single valid transition is sufficient for correctness, but additional transitions may improve mixing. For illustration, the wrapped sampler below uses `burnin = 0` and `nsteps = 2`. Because **fmcmc** counts the supplied initial state as its first stored step, this call performs one Markov transition and returns the second state during each outer **dapper** iteration.

```
posterior_f <- function(dmat, theta) {
  one_step <- MCMC(
    initial = theta,
    fun     = log_latent_posterior,
    nsteps  = 2,
    burnin  = 0,
    kernel  = kernel_normal(scale = 0.2),
    progress = FALSE,
    x = dmat[, -1],
    y = dmat[, 1]
  )

  one_step[2,]
}
```

Finally, to get **dapper** working with the new sampler, all we need to do is swap out the definition for `posterior_f()` in the privacy model object and rerun our code. We run one outer chain for 25,000 total MCMC iterations, discard the first 1,000 as warmup, and retain 24,000 post-warmup draws under the new *t*-distribution prior. A summary of these draws is given below.

```
#> # A tibble: 3 x 10
#>   variable  mean median    sd  mad    q5   q95  rhat ess_bulk ess_tail
#>   <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>    <dbl>    <dbl>
#> 1 beta0   -0.765 -0.551  1.40  1.13 -3.36  1.09  1.05     37.0     25.2
#> 2 beta1   -1.68  -1.88  1.46  1.49 -3.66  0.749 1.00     31.4     35.3
#> 3 beta2    0.951  0.932  1.06  1.17 -0.686 2.60  1.07     41.0     88.7
```

6 Computational considerations

The computational performance of **dapper** depends on both the cost of each MCMC iteration and the number of iterations required to obtain reliable posterior estimates. We first report indicative runtimes and effective sample sizes for the examples in this article. We then discuss how the privacy loss budget affects mixing and provide practical guidance for diagnosing inefficient chains.

6.1 Empirical performance of the examples

Table 4 reports single-run wall-clock times for the configurations used in the examples. The measurements were obtained on an eight-core Apple M1 Pro using R 4.6.1 (R Core Team, 2026) and `dapper` 1.1.1. The randomized-response example uses four chains on four parallel workers to demonstrate parallel execution and multi-chain diagnostics. The remaining examples use one chain and one worker; these settings were selected in part to keep the demonstrations reasonably similar in wall-clock duration.

Table 4: Indicative wall-clock runtimes and minimum bulk effective sample sizes for the article examples. Minimum bulk ESS is taken across the model parameters. Under the configurations shown, all four examples finish in under 70 seconds, but their minimum bulk effective sample sizes vary substantially.

Example	Records	Chains	Iterations (warmup)	Retained draws	Elapsed (s)	Min. bulk ESS
Randomized response	400	4	6,000 (1,000)	20,000	17.1	282.2
Discrete Gaussian	400	1	2,000 (1,000)	1,000	68.9	450.2
Conjugate regression	50	1	25,000 (1,000)	24,000	62.2	133.3
Wrapper regression	50	1	25,000 (1,000)	24,000	69.3	31.4

The wrapper regression is included to show the overhead associated with calling an existing MCMC implementation from within `dapper`. Its inner sampler performs only one transition per outer iteration, so the increased time when compared to the conjugate implementation likely comes from function-call overhead. More generally, one `dapper` iteration consists of a posterior update, simulation of a candidate latent data set, and a record-level Metropolis update for each of the n latent records. Consequently, total work is approximately linear in n iter, the number of records, and the number of chains, although independent chains can be distributed across workers. The constants depend on the cost of the user-supplied `posterior_f()`, `latent_f()`, `statistic_f()`, and `mechanism_f()` functions; in particular, wrapping a more expensive posterior sampler increases the cost of every outer iteration.

The single inner transition used in Example 4 is sufficient for correctness, but increasing the number of inner transitions per call may improve mixing. This may be especially worthwhile when updating θ conditional on the latent data is inexpensive relative to updating the latent data. The number of inner transitions is therefore a computational setting that can be chosen using pilot runs and comparisons of effective sample size per unit of computation.

In practice, users should begin with a shorter pilot run and use standard MCMC diagnostic tools such as trace plots, $\hat{R}$, bulk and tail ESS, Monte Carlo standard errors, and acceptance rates to determine whether additional iterations are needed. Multiple chains from dispersed initial values are preferable when resources permit, and can be run in parallel using `furr`. The outer warmup period should be long enough to remove initialization effects, while inner burn-in within `posterior_f()` should generally be omitted or kept short because of the cost incurred during every outer iteration.

6.2 Mixing and privacy loss budget

Mixing can be poor when the posterior under a given privacy mechanism is much more dispersed than the posterior that would arise using the confidential data. In other words, a small privacy loss budget can result in poor mixing. Intuitively, this issue arises because the mixing of our Gibbs sampler is constrained by the coupling between the parameter and the confidential data. Decreasing the privacy budget will increase posterior uncertainty, until eventually the chain step sizes become too small to effectively explore the private posterior. The rest of this section explores a toy example that will provide insight into this phenomenon.

Suppose the confidential data set consists of a single observation $x \in \mathbb{R}$, and consider the scenario where a user makes a request to view x and in return receives $s := x + v$, which is a noise-infused version of x . For simplicity, we do not worry about constructing an ϵ -DP mechanism, and simply take $v \sim N(0, \epsilon^{-2})$ for some $\epsilon > 0$. However, it will still be useful to think of ϵ as the privacy loss budget, since smaller values of ϵ correspond to a larger amount of noise. Using a flat prior and a normally distributed likelihood results in a normally distributed posterior described below.

$$\begin{aligned} f(\theta) &\propto 1 \\ s \mid x &\sim N(x, \epsilon^{-2}) \\ x \mid \theta &\sim N(\theta, \sigma^2) \end{aligned}$$

With the above model, the data augmentation MCMC process consists of the following two steps:

- Step 1: Sample from $x \mid \theta, s \sim N(\mu, \tau^2)$, where μ and τ^2 are given by

$$\mu := \frac{s/\epsilon^{-2} + \theta/\sigma^2}{1/\epsilon^{-2} + 1/\sigma^2}$$

$$\tau^2 := \frac{1}{1/\epsilon^{-2} + 1/\sigma^2}.$$

- Step 2: Sample from $\theta \mid x, s \sim N(x, \sigma^2)$.

In the setting of this example, [Liu and Wu \(1999\)](#) showed the Bayesian fraction of missing information gives the exact convergence rate. The Bayesian fraction of missing information, γ , is defined as

$$\gamma := 1 - \frac{E[\text{Var}(\theta \mid s, x) \mid s]}{\text{Var}(\theta \mid s)} = 1 - \frac{E[\text{Var}(\theta \mid x)]}{\text{Var}(\theta \mid s)}.$$

Plugging the appropriate quantities into the above panel gives us

$$\gamma = 1 - \frac{\sigma^2}{\sigma^2 + \epsilon^{-2}} = 1 - \frac{1}{1 + \epsilon^{-2}/\sigma^2}.$$

The chain converges faster as $\gamma \rightarrow 0$ and slower as $\gamma \rightarrow 1$. From the right-hand term in the above panel, we can see that γ depends only on ϵ^{-2}/σ^2 , and as the privacy loss budget decreases (i.e., more noise is being added to x), $\gamma \rightarrow 1$.

To illustrate this computational effect in a complete example, we repeated the conjugate regression analysis from Example 3 over five privacy loss budgets. We hold the confidential data and a single unit-scale Laplace noise draw fixed. This single draw is used to produce the private release for all budgets. Using a common noise source allows us to reduce extraneous variation between the runs. Each run uses four chains, four workers, 6,000 total iterations per chain, and 1,000 warmup iterations per chain, leaving 5,000 retained post-warmup draws per chain and 20,000 pooled draws for each privacy budget. The four chains are used here so that $\hat{R}$ can be calculated. The original example uses one chain because its purpose is primarily illustrative.

Table 5: Conjugate-regression mixing diagnostics across Laplace privacy loss budgets using coupled noise draws and a fixed MCMC configuration. Minimum bulk and tail ESS and maximum R-hat are taken across the model parameters. This experiment shows that the privacy loss budget can substantially affect mixing.

Total epsilon	Laplace scale	Elapsed (s)	Min. bulk ESS	Min. tail ESS	Bulk ESS / s	Max. R-hat
2.5	6.000	15.8	74.4	182.2	4.7	1.056
5	3.000	15.4	122.3	268.5	7.9	1.035
10 (Example 3)	1.500	15.3	163.7	274.7	10.7	1.018
20	0.750	15.2	240.8	479.6	15.9	1.011
40	0.375	15.2	417.9	743.2	27.5	1.008

As Table 5 shows, elapsed time remains nearly constant across these runs, whereas the number of effectively independent draws changes substantially. Over the range considered, the minimum bulk ESS increases from 74.4 to 417.9 and the maximum $\hat{R}$ decreases from 1.056 to 1.008 as the privacy loss budget increases from 2.5 to 40. The value used in Example 3, $\epsilon = 10$, yields a minimum bulk ESS of 163.7 and a maximum $\hat{R}$ of 1.018. These diagnostic patterns are consistent with better mixing at larger budgets.

This comparison is restricted to one privacy mechanism and one model, so it should be interpreted with caution. Although ϵ has a common interpretation as a worst-case privacy-loss bound, its numerical value is not a global scale for MCMC performance. At the same ϵ , a sampler may mix well under one privacy mechanism but poorly under another. As such, privacy-budget sensitivity checks should be interpreted within the specific mechanism, statistic, and model being analyzed.

If one cannot achieve good mixing with acceptable compute times, we recommend artificially varying the privacy budget to see if mixing significantly improves. If this is the case, then [dapper](#) may be poorly suited for the problem at hand, and alternative methods, such as the pseudo-marginal method ([Andrieu and Roberts, 2009](#)), can be pursued. Additionally, since programming samplers can be a bug-prone endeavor, it may be helpful to artificially vary the privacy loss budget as a sanity check; slow mixing could be due to an error introduced when specifying the components of [dapper](#). For example, one can generate synthetic data using `latent_f()` and compare the posterior draws from `posterior_f()` and `dapper_sample()`. Generally speaking, results should look similar when using a sufficiently large privacy loss budget.

7 Summary

Differential privacy powers many state-of-the-art data privacy implementations. Software supporting DP deployment and analysis is growing rapidly, with several large companies building software ecosystems for working with differential privacy (e.g., AWS Clean Rooms (Amazon Web Services, 2024), SmartNoise (Microsoft and OpenDP, 2024), and PipelineDP (Google Anonymization Team and OpenMined, 2022)). However, the majority of these software tools only address privacy and not the ensuing analysis or, if they do, only address the analysis for specific models. Furthermore, none of the mentioned software suites are well suited for R users.

dapper helps fill a capability gap by providing researchers a way to perform privacy-aware Bayesian inference in R. As summarized in Table 1, most related tools construct private releases or fit models directly to confidential data. **SimBaRepro** is an important exception that also supports inference from already privatized data, but provides frequentist tests and confidence sets rather than posterior draws or a point estimate.

This package offers a significant step forward in extending general-purpose statistical inference tools from confidential to privatized data. Despite its strengths, there are several cumbersome requirements for good performance that limit its potential. First, the privacy mechanism's log density or log probability mass must be evaluable. Second, a record-additive statistic must be used to leverage **dapper**'s full computational potential. Third, the non-private posterior sampler needs to mix well. Finally, a very small privacy loss budget can result in excessively slow mixing. Future versions of **dapper** could incorporate methodological developments that relax some of these requirements.

Supporting software

Data preparation uses **tidyverse** (Wickham et al., 2019), including **dplyr** (Wickham et al., 2026), **tibble** (Müller and Wickham, 2026), and **tidyr** (Wickham et al., 2025b). Figures use **ggplot2** (Wickham, 2016), **ggsci** (Xiao, 2026), and **scales** (Wickham et al., 2025a), alongside the diagnostic plotting tools described above. The examples also use **MASS** (Venables and Ripley, 2002) for multivariate normal simulation and **VGAM** (Yee, 2025) for Laplace densities and simulation. Parallel execution uses the **future** framework (Bengtsson, 2021) through **furrr**.

The article is generated with **rmarkdown** (Allaire et al., 2026), **knitr** (Xie, 2025), and **rjtools** (O'Hara-Wild et al., 2025), with **kableExtra** (Zhu, 2026) for table formatting.

Disclosure statement

Kevin Eng and Ruobin Gong are grateful for support from the Alfred P. Sloan Foundation. Jordan Awan's research was supported in part by NSF grants SES-2150615 and SES-2610910. Nianqiao Ju acknowledges support from the NSF grant DMS-2610236. OpenAI's Codex was used for light editing and assistance with selected tables and figures. All outputs were reviewed and verified by the authors.

References

- J. Abowd, R. Ashmead, R. Cumings-Menon, S. Garfinkel, M. Heineck, C. Heiss, R. Johns, D. Kifer, P. Leclerc, A. Machanavajjhala, B. Moran, W. Sexton, M. Spence, and P. Zhuravlev. The 2020 Census Disclosure Avoidance System TopDown algorithm. *Harvard Data Science Review*, (Special Issue 2), June 2022. doi: 10.1162/99608f92.529e3cb9. URL <http://dx.doi.org/10.1162/99608f92.529e3cb9>. [p7]
- J. M. Abowd. The U.S. Census Bureau adopts differential privacy. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '18, page 2867. ACM, July 2018. doi: 10.1145/3219819.3226070. URL <http://dx.doi.org/10.1145/3219819.3226070>. [p1]
- J. Allaire, Y. Xie, C. Dervieux, J. McPherson, J. Luraschi, K. Ushey, A. Atkins, H. Wickham, J. Cheng, W. Chang, and R. Iannone. *rmarkdown: Dynamic Documents for R*, 2026. URL <https://doi.org/10.32614/CRAN.package.rmarkdown>. R package version 2.32. [p24]
- Amazon Web Services. *AWS Clean Rooms User Guide*. Amazon Web Services, Inc., 2024. URL <https://docs.aws.amazon.com/clean-rooms/latest/userguide/>. Accessed: 2024-05-22. [p24]

- C. Andrieu and G. O. Roberts. The pseudo-marginal approach for efficient Monte Carlo computations. *The Annals of Statistics*, 37(2):697–725, Apr. 2009. ISSN 0090-5364. doi: 10.1214/07-aos574. URL <http://dx.doi.org/10.1214/07-AOS574>. [p23]
- J. Awan and A. Slavković. Structure and sensitivity in differential privacy: Comparing K-Norm mechanisms. *Journal of the American Statistical Association*, 116(534):935–954, Apr. 2021. ISSN 1537-274X. doi: 10.1080/01621459.2020.1773831. URL <http://dx.doi.org/10.1080/01621459.2020.1773831>. [p15]
- J. Awan and Z. Wang. Simulation-based, finite-sample inference for privatized data. *Journal of the American Statistical Association*, 120(551):1669–1682, Jan. 2025. ISSN 1537-274X. doi: 10.1080/01621459.2024.2427436. URL <http://dx.doi.org/10.1080/01621459.2024.2427436>. [p2]
- R. F. Barber and J. C. Duchi. Privacy and statistical risk: Formalisms and minimax bounds, 2014. URL <https://arxiv.org/abs/1412.4451>. [p12]
- H. Bengtsson. A unifying framework for parallel and distributed processing in R using futures. *The R Journal*, 13(2):208–227, 2021. doi: 10.32614/RJ-2021-048. URL <https://doi.org/10.32614/RJ-2021-048>. [p24]
- H. Bengtsson. *progressr: An Inclusive, Unifying API for Progress Updates*, 2026. URL <https://doi.org/10.32614/CRAN.package.progressr>. R package version 1.0.0. [p6]
- P. J. Bickel, E. A. Hammel, and J. W. O’Connell. Sex bias in graduate admissions: Data from Berkeley. *Science*, 187(4175):398–404, Feb. 1975. ISSN 1095-9203. doi: 10.1126/science.187.4175.398. URL <http://dx.doi.org/10.1126/science.187.4175.398>. [p7]
- H. W. Borchers. *pracma: Practical Numerical Math Functions*, 2025. URL <https://doi.org/10.32614/CRAN.package.pracma>. R package version 2.4.6. [p17]
- M. Bun and T. Steinke. Concentrated differential privacy: Simplifications, extensions, and lower bounds. In M. Hirt and A. Smith, editors, *Theory of Cryptography – TCC 2016-B*, volume 9985 of *Lecture Notes in Computer Science*, pages 635–658. Springer, 2016. doi: 10.1007/978-3-662-53641-4_24. URL https://doi.org/10.1007/978-3-662-53641-4_24. [p3, 12]
- P.-C. Bürkner, J. Gabry, M. Kay, and A. Vehtari. posterior: Tools for working with posterior distributions, 2026. URL <https://doi.org/10.32614/CRAN.package.posterior>. R package version 1.7.0. [p7]
- C. Canonne, G. Kamath, and T. Steinke. Discrete Gaussian for differential privacy. *Journal of Privacy and Confidentiality*, 12(1), July 2022. ISSN 2575-8527. doi: 10.29012/jpc.784. URL <http://dx.doi.org/10.29012/jpc.784>. [p7]
- T. Dalenius and S. P. Reiss. Data-swapping: A technique for disclosure control. *Journal of Statistical Planning and Inference*, 6(1):73–85, 1982. ISSN 0378-3758. doi: 10.1016/0378-3758(82)90058-1. URL [http://dx.doi.org/10.1016/0378-3758\(82\)90058-1](http://dx.doi.org/10.1016/0378-3758(82)90058-1). [p3]
- B. Ding, J. Kulkarni, and S. Yekhanin. Collecting telemetry data privately. In *Advances in Neural Information Processing Systems*, volume 30, pages 3571–3580. Curran Associates, Inc., 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/253614bbac999b38b5b60cae531c4969-Abstract.html>. [p1]
- X. Du, Z. Wang, and J. Awan. *SimBaRepro: Simulation-Based, Finite-Sample Inference via Repro Samples*, 2025. URL <https://CRAN.R-project.org/package=SimBaRepro>. R package version 0.1.0. [p2]
- J. C. Duchi, M. I. Jordan, and M. J. Wainwright. Minimax optimal procedures for locally private estimation. *Journal of the American Statistical Association*, 113(521):182–201, 2018. doi: 10.1080/01621459.2017.1389735. URL <https://doi.org/10.1080/01621459.2017.1389735>. [p12]
- C. Dwork, K. Kenthapadi, F. McSherry, I. Mironov, and M. Naor. Our data, ourselves: Privacy via distributed noise generation. In S. Vaudenay, editor, *Advances in Cryptology – EUROCRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages 486–503. Springer Berlin Heidelberg, 2006a. ISBN 9783540345473. doi: 10.1007/11761679_29. URL http://dx.doi.org/10.1007/11761679_29. [p3]
- C. Dwork, F. McSherry, K. Nissim, and A. Smith. Calibrating noise to sensitivity in private data analysis. In S. Halevi and T. Rabin, editors, *Theory of Cryptography*, volume 3876 of *Lecture Notes in Computer Science*, pages 265–284. Springer Berlin Heidelberg, 2006b. ISBN 9783540327325. doi: 10.1007/11681878_14. URL http://dx.doi.org/10.1007/11681878_14. [p1, 3]

- Ú. Erlingsson, V. Pihur, and A. Korolova. RAPPOR: Randomized aggregatable privacy-preserving ordinal response. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, CCS'14*, pages 1054–1067. ACM, Nov. 2014. ISBN 978-1-4503-2957-6. doi: 10.1145/2660267.2660348. URL <http://dx.doi.org/10.1145/2660267.2660348>. [p1]
- J. Gabry and T. Mahr. bayesplot: Plotting for Bayesian models, 2026. URL <https://doi.org/10.32614/CRAN.package.bayesplot>. R package version 1.16.0. [p7]
- A. Gelman, A. Jakulin, M. G. Pittau, and Y.-S. Su. A weakly informative default prior distribution for logistic and other regression models. *The Annals of Applied Statistics*, 2(4):1360–1383, 2008. doi: 10.1214/08-AOAS191. URL <https://doi.org/10.1214/08-AOAS191>. [p20]
- S. Giddens. *DPpack: Differentially Private Statistical Analysis and Machine Learning*, 2024. URL <https://CRAN.R-project.org/package=DPpack>. R package version 0.2.2. [p2]
- R. Gong. Transparent privacy is principled privacy. *Harvard Data Science Review*, (Special Issue 2), June 2022. doi: 10.1162/99608f92.b5d3faaa. URL <http://dx.doi.org/10.1162/99608f92.b5d3faaa>. [p1]
- Google Anonymization Team and OpenMined. PipelineDP: An open-source framework for differentially private data processing, 2022. URL <https://github.com/OpenMined/PipelineDP>. Version 0.1.0-beta. [p24]
- N. J. Higham. Computing a nearest symmetric positive semidefinite matrix. *Linear Algebra and its Applications*, 103:103–118, May 1988. ISSN 0024-3795. doi: 10.1016/0024-3795(88)90223-6. URL [http://dx.doi.org/10.1016/0024-3795\(88\)90223-6](http://dx.doi.org/10.1016/0024-3795(88)90223-6). [p17]
- N. Holohan, S. Braghin, P. Mac Aonghusa, and K. Levacher. diffprivlib: The IBM differential privacy library. *arXiv e-prints*, 1907.02444 [cs.CR], July 2019. doi: 10.48550/arXiv.1907.02444. URL <https://doi.org/10.48550/arXiv.1907.02444>. [p2]
- N. Ju, J. Awan, R. Gong, and V. Rao. Data augmentation MCMC for Bayesian inference from privatized data. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. H. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 12732–12743. Curran Associates, Inc., 2022. doi: 10.52202/068431-0925. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/529d0f9b0fb7c8d4b7d52221faee48d6-Abstract-Conference.html. [p1, 2, 4, 14, 15]
- A. F. Karr. Data sharing and access. *Annual Review of Statistics and Its Application*, 3(1):113–132, June 2016. ISSN 2326-831X. doi: 10.1146/annurev-statistics-041715-033438. URL <http://dx.doi.org/10.1146/annurev-statistics-041715-033438>. [p3]
- V. Karwa, D. Kifer, and A. B. Slavković. Private posterior distributions from variational approximations, 2015. URL <https://arxiv.org/abs/1511.07896>. [p1]
- C. T. Kenny, S. Kuriwaki, C. McCartan, E. T. R. Rosenman, T. Simko, and K. Imai. The use of differential privacy for census data and its impact on redistricting: The case of the 2020 U.S. Census. *Science Advances*, 7(41):eabk3283, Oct. 2021. ISSN 2375-2548. doi: 10.1126/sciadv.abk3283. URL <http://dx.doi.org/10.1126/sciadv.abk3283>. [p1]
- J. S. Liu and Y. N. Wu. Parameter expansion for data augmentation. *Journal of the American Statistical Association*, 94(448):1264–1274, 1999. doi: 10.1080/01621459.1999.10473879. URL <https://www.tandfonline.com/doi/abs/10.1080/01621459.1999.10473879>. [p23]
- Microsoft and OpenDP. SmartNoise SDK: Tools for differential privacy on tabular data, 2024. URL <https://github.com/opendp/smartnoise-sdk>. Accessed: 2024-05-22. [p24]
- I. Mironov. On significance of the least significant bits for differential privacy. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security, CCS '12*, pages 650–661, New York, NY, USA, 2012. Association for Computing Machinery. ISBN 9781450316514. doi: 10.1145/2382196.2382264. URL <https://doi.org/10.1145/2382196.2382264>. [p7]
- K. Müller and H. Wickham. *tibble: Simple Data Frames*, 2026. URL <https://doi.org/10.32614/CRAN.package.tibble>. R package version 3.3.1. [p24]
- M. O'Hara-Wild, S. Kobakian, H. S. Zhang, D. Cook, S. Urbanek, and C. Dervieux. *rjtools: Preparing, Checking, and Submitting Articles to the 'R Journal'*, 2025. URL <https://doi.org/10.32614/CRAN.package.rjtools>. R package version 1.0.18.1. [p24]

- M. Plummer, N. Best, K. Cowles, and K. Vines. CODA: Convergence diagnosis and output analysis for MCMC. *R News*, 6(1):7–11, 2006. URL https://www.r-project.org/doc/Rnews/Rnews_2006-1.pdf. [p20]
- L. Prediger, N. Loppi, S. Kaski, and A. Honkela. d3p – a Python package for differentially-private probabilistic programming. *Proceedings on Privacy Enhancing Technologies*, 2022(2):407–425, 2022. doi: 10.2478/popets-2022-0052. URL <https://doi.org/10.2478/popets-2022-0052>. [p2]
- R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2026. URL <https://doi.org/10.32614/R.manuals>. [p22]
- C. P. Robert and G. Casella. *Monte Carlo Statistical Methods*. Springer Texts in Statistics. Springer New York, 2 edition, 2004. ISBN 9781475741452. doi: 10.1007/978-1-4757-4145-2. URL <http://dx.doi.org/10.1007/978-1-4757-4145-2>. [p4]
- B. I. P. Rubinstein and F. Aldà. Pain-free random differential privacy with sensitivity sampling. In D. Precup and Y. W. Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 2950–2959. PMLR, 2017. URL <https://proceedings.mlr.press/v70/rubinstein17a.html>. [p2]
- A. R. Santos-Lozada, J. T. Howard, and A. M. Verdery. How differential privacy will affect our understanding of health disparities in the United States. *Proceedings of the National Academy of Sciences*, 117(24):13405–13412, May 2020. ISSN 1091-6490. doi: 10.1073/pnas.2003714117. URL <http://dx.doi.org/10.1073/pnas.2003714117>. [p1]
- A. Scheidegger. *adaptMCMC: Implementation of a Generic Adaptive Monte Carlo Markov Chain Sampler*, 2024. URL <https://CRAN.R-project.org/package=adaptMCMC>. R package version 1.5. [p5]
- Stan Development Team. RStan: the R interface to Stan, 2025. URL <https://mc-stan.org/>. R package version 2.32.7. [p1]
- J. Tang, A. Korolova, X. Bai, X. Wang, and X. Wang. Privacy loss in Apple’s implementation of differential privacy on MacOS 10.12, 2017. URL <https://doi.org/10.48550/arXiv.1709.02753>. [p1]
- T. Tarler. *tidydp: Tidy Differential Privacy*, 2025. URL <https://CRAN.R-project.org/package=tidydp>. R package version 0.1.0. [p2]
- The OpenDP Project. *opendp: R Bindings for the OpenDP Library*, 2026. URL <https://opendp.org/>. R package version 0.15.1. [p2]
- Tumult Labs. SafeTab: DP algorithms for 2020 Census detailed DHC race & ethnicity. Technical report, Tumult Labs, 2022. URL <https://www2.census.gov/about/partners/cac/sac/meetings/2022-03/dhc-attachment-1-safetab-dp-algorithms.pdf>. [p7]
- D. Vaughan, H. Bengtsson, and M. Dancho. *furrr: Apply Mapping Functions in Parallel using Futures*, 2026. URL <https://doi.org/10.32614/CRAN.package.furrr>. R package version 0.4.0. [p6]
- G. G. Vega Yon and P. Marjoram. fmcmc: A friendly MCMC framework. *Journal of Open Source Software*, 4(39):1427, jul 2019. doi: 10.21105/joss.01427. URL <https://doi.org/10.21105/joss.01427>. [p5]
- W. N. Venables and B. D. Ripley. *Modern Applied Statistics with S*. Springer, New York, fourth edition, 2002. doi: 10.1007/978-0-387-21706-2. URL <https://doi.org/10.1007/978-0-387-21706-2>. ISBN 0-387-95457-0. [p24]
- Y. Wang, D. Kifer, J. Lee, and V. Karwa. Statistical approximating distributions under differential privacy. *Journal of Privacy and Confidentiality*, 8(1), Dec. 2018. ISSN 2575-8527. doi: 10.29012/jpc.666. URL <http://dx.doi.org/10.29012/jpc.666>. [p1]
- S. L. Warner. Randomized response: A survey technique for eliminating evasive answer bias. *Journal of the American Statistical Association*, 60(309):63–69, Mar. 1965. ISSN 1537-274X. doi: 10.1080/01621459.1965.10480775. URL <http://dx.doi.org/10.1080/01621459.1965.10480775>. [p8]
- H. Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016. ISBN 978-3-319-24277-4. doi: 10.1007/978-3-319-24277-4. URL <https://doi.org/10.1007/978-3-319-24277-4>. [p24]

- H. Wickham, M. Averick, J. Bryan, W. Chang, L. D. McGowan, R. François, G. Golemund, A. Hayes, L. Henry, J. Hester, M. Kuhn, T. L. Pedersen, E. Miller, S. M. Bache, K. Müller, J. Ooms, D. Robinson, D. P. Seidel, V. Spinu, K. Takahashi, D. Vaughan, C. Wilke, K. Woo, and H. Yutani. Welcome to the tidyverse. *Journal of Open Source Software*, 4(43):1686, 2019. doi: 10.21105/joss.01686. URL <https://doi.org/10.21105/joss.01686>. [p24]
- H. Wickham, T. L. Pedersen, and D. Seidel. *scales: Scale Functions for Visualization*, 2025a. URL <https://doi.org/10.32614/CRAN.package.scales>. R package version 1.4.0. [p24]
- H. Wickham, D. Vaughan, and M. Girlich. *tidyr: Tidy Messy Data*, 2025b. URL <https://doi.org/10.32614/CRAN.package.tidyr>. R package version 1.3.2. [p24]
- H. Wickham, R. François, L. Henry, K. Müller, and D. Vaughan. *dplyr: A Grammar of Data Manipulation*, 2026. URL <https://doi.org/10.32614/CRAN.package.dplyr>. R package version 1.2.1. [p24]
- O. Williams and F. McSherry. Probabilistic inference and differential privacy. In J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta, editors, *Advances in Neural Information Processing Systems*, volume 23, pages 2451–2459. Curran Associates, Inc., 2010. URL https://proceedings.neurips.cc/paper_files/paper/2010/file/fb60d411a5c5b72b2e7d3527cfc84fd0-Paper.pdf. [p1, 4]
- R. L. Winkler, J. L. Butler, K. J. Curtis, and D. Egan-Robertson. Differential privacy and the accuracy of county-level net migration estimates. *Population Research and Policy Review*, 41(2):417–435, Apr. 2022. ISSN 1573-7829. doi: 10.1007/s11113-021-09664-5. URL <http://dx.doi.org/10.1007/s11113-021-09664-5>. [p1]
- N. Xiao. *ggsci: Scientific Journal and Sci-Fi Themed Color Palettes for 'ggplot2'*, 2026. URL <https://doi.org/10.32614/CRAN.package.ggsci>. R package version 5.2.0. [p24]
- Y. Xie. *knitr: A General-Purpose Package for Dynamic Report Generation in R*, 2025. URL <https://doi.org/10.32614/CRAN.package.knitr>. R package version 1.51. [p24]
- T. W. Yee. *VGAM: Vector Generalized Linear and Additive Models*, 2025. URL <https://doi.org/10.32614/CRAN.package.VGAM>. R package version 1.1-14. [p24]
- H. Zhu. *kableExtra: Construct Complex Table with 'kable' and Pipe Syntax*, 2026. URL <https://doi.org/10.32614/CRAN.package.kableExtra>. R package version 1.4.1. [p24]

Kevin Eng
Rutgers University
Department of Statistics
Piscataway, NJ 08854
ke157@stat.rutgers.edu

Jordan A. Awan
University of Pittsburgh
Department of Statistics
Pittsburgh, PA 15260
<https://jordan-awan.com/>
jaa557@pitt.edu

Nianqiao Phyllis Ju
Dartmouth
Department of Mathematics
Hanover, NH 03755
<https://nianqiaoju.github.io/>
nianqiao.ju@dartmouth.edu

Vinayak A. Rao
Purdue University
Department of Statistics
West Lafayette, IN 47907
<https://varao.github.io/>
varao@purdue.edu

Ruobin Gong
Rutgers University

Department of Statistics
Piscataway, NJ 08854
<https://ruobingong.github.io/>
ruobin.gong@rutgers.edu